CYRUS SCATTER / INDEPENDENT RESEARCH

Houdini & tyFlow: Practical Vegetation Workflows

A CyrusScatter companion study: rooted grass, two-stone contact, rigid pebbles and rigid versus deforming leaves

24 October 2026 · Evidence baseline: Houdini 22 / tyFlow 2.101

A reproducible workflow study with exact inputs, node and operator order, separate root-position and root-orientation controls, initialization, reset, caching, rendering, export and instance behavior.

The central distinction is between static guide correction, simulation that runs internally at one evaluated frame, and a timeline simulation whose accepted result is frozen. The report keeps those paths separate and ends with a proposed Cyrus artist workflow.

Official documentation and inspected example assets support the recipes. No Houdini, 3ds Max, tyFlow simulation or renderer test was executed. Numeric settings are proposed starting values.

Contents

Four consistent scene examples, paired application recipes, explicit preparation and lifecycle steps, and a proposed Cyrus artist workflow.

Evidence and version boundaries . . . 3
Common fixtures and reproduction rules . . . 7
Houdini: grass and one stone . . . 10
Houdini: grass between two stones . . . 15
Houdini: rigid pebbles . . . 17
Houdini: fallen leaves at a curb . . . 19
tyFlow: grass and one stone . . . 22
tyFlow: grass between two stones . . . 31
tyFlow: rigid pebbles . . . 32
tyFlow: fallen leaves at a curb . . . 35
Reset, cache, render and export . . . 40
Artist work, failures and local acceptance . . . 45
Evidence register and source links . . . 51
Proposed Cyrus artist workflow . . . 62

Reading the evidence

D = official documentation; I = example file structure inspected; P = proposed recipe assembled from documented components; X = executed application test. No X-level application test was possible. Bracketed source IDs link to the source register.

SECTION 1

Evidence and version boundaries

This is a practical companion to the previous CyrusScatter vegetation study. It examines four repeatable scene situations in Houdini and tyFlow: rooted grass beside one stone, rooted grass between two stones, rigid pebbles on uneven ground, and fallen leaves at a curb. The leaf case has separate rigid and deforming recipes. It does not repeat the earlier numerical-method survey or infer proprietary algorithms from an interface.

Main finding: Houdini offers a documented static guide-collision operation and a Vellum route that can return a settled pose through an internal roll-forward. tyFlow offers documented binding, cloth and rigid-body simulation components, but its history-independent setting is not a documented equilibrium-fitting command. A held simulation frame is a static deliverable obtained through simulation; that distinction remains visible throughout this report. [S01] [S03] [S54]

Evidence labels

Label	Meaning in this study	What it establishes
D - documented	Official parameter or workflow documentation was read.	A public capability or control, subject to the stated version.
I - inspected example	An official downloadable example's network and parameter records were examined without launching the application.	The saved example's structure and selected settings; no runtime success.
P - proposed recipe	We assembled a complete scene recipe from documented components and chose test values.	A reproducible local-validation procedure; its outcome has not been executed here.
X - executed application test	The scene was evaluated in Houdini or Max/tyFlow, rendered, or round-tripped.	**None in this study.** Neither Houdini/hython nor 3ds Max/tyFlow was available in the execution environment.

Every complete scene assembly below is **P**, supported by **D** and, where identified, **I**. Node order, input wiring, initialization and acceptance criteria are explicit. Numeric presets are proposed starting points. No simulation timing, frame count to convergence, root-error measurement, collision-free result, renderer match or export round-trip is reported as an executed result.

Version baseline and conflicts

Item	Verified baseline	Consequence for reproduction
Houdini documentation	Current pages identify Houdini 22.0.	Follow the H22 UI names in this report. Earlier node availability is not proof that every H22 control exists in H21.
H22 hair orientation solve	The new Orientation Solve = Quasistatic and Angular Damping are explicitly listed as H22 additions.	These are optional H22 branches. The baseline recipe keeps the traditional Dynamic orientation method. [S05]
Guide Deform	The current redesigned node is marked Since 22.0.	Do not back-port its current guide capture/Preserve Shape options to H21 by name alone. [S06]
tyFlow / Max	Reproduce with tyFlow 2.101 (8 October 2026). Max 2027 support began in 2.004.	Record the installed tyFlow build on the authoring machine and workers. [S31]
Current CUDA path	Release history specifies CUDA 12.8; the download page identifies matching supplemental modules for 2.005+.	Older FAQ/system pages say 10.0/11.2. Use the current package and driver requirements. Since 1.132 the supplemental DLLs are optional for CCCS, with reduced performance when absent. [S31] [S64] [S35]
tyFlow license	FREE disables GPU acceleration, general particle multithreading and tyCache export.	The CCCS recipes and tyCache-writing steps require PRO; the CPU rigid method can be investigated in FREE with a different output route. [S33]

The version-history corrections also matter when reading older advice: it adds SDF hulls and tyCache subframe data in 2.000, and records later support that conflicts with older sticky-penetration and cache-motion-blur warnings. The recipes avoid relying on those disputed corner cases. The report claims only the documented controls actually used; it does not equate an upstream PhysX feature with an exposed tyFlow workflow. [S31]

Decisions that prevent misleading recipes

- **Root position and root orientation are separate.** A fixed point can still be a hinge. Two fixed collinear samples establish a base tangent but leave axial roll unconstrained. A noncollinear fixed patch establishes a base frame; the distinction is geometric, not a claim about a hidden solver.
- **Guide clearance and visible-mesh clearance are separate.** A collision-free curve can drive a ribbon whose edges still touch a stone. Inspect the decoded mesh before accepting a pose.
- **Guide deformation is separate from collision solving.** H22 Guide Deform's shape-preservation controls do not document an obstacle-contact input. Standard Point Deform explicitly warns about wire-like deformation lattices. For the prepared straight blade, use Path Deform; for an arbitrary bound curved blade, the documented Wire Capture/Wire Deform route needs stable correspondence and frames. [S06] [S13] [S10] [S79]
- **Hair Bind in tyFlow means attachment to Ornatrix strands.** It requires Position Hair data. The guide simulation recipe instead uses Particle Bind/Angle Bind; the surface alternative uses Cloth Bind. [S41]

- **Collision shape and render shape are separate.** Show the actual rigid hull. A whole terrain convex hull can bridge depressions; a convex leaf proxy can bridge a curled underside. These are shape-choice consequences, not friction problems.
- **A shared source asset does not imply shared deformed output.** Rigid instances can reuse geometry while their transforms change. Differently bent blades and leaves generally have different visible vertex positions. The report identifies which documented output modes preserve or flatten the instance representation.

What was inspected in example files

The official HairOrient, BasicWireCapture and TwistWireDeform assets were downloaded and their saved network records inspected. HairOrient supplies useful orientation preparation, but its saved setup uses **soft roots and collisions disabled**; it is not a stone-contact or hard-root demonstration. BasicWireCapture documents the capture/deform relationship, and TwistWireDeform records tangent-plus-normal frame transfer. The evidence register includes official links and file hashes. Additional inspected Path Deform and RBD examples are described there. [S77] [S78] [S79]

The tyFlow download page advertises an official example archive dated 2 September 2026. Historical named examples are useful discovery leads, but no matching .max scene was opened or simulated here. A documented archive listing is not evidence that a particular grass/stone or leaf/curb recipe was demonstrated. [S64]

Houdini node availability and overloaded terms

Capability	Documented version / limitation
Guide Collide With VDB	Since 16.0. Static SOP; extra collision VDBs enter input 4.
Vellum Constraints / Solver	Since 17.0. Hair with bend/twist, positional pins and orientation pins; dynamic and frame-local quasistatic simulation modes.
RBD Configure / RBD Bullet Solver SOP	Since 18.0. Packed render geometry, optional proxy, separate external collision input.
Path Deform	Node since 18.0. Matching piece names is demonstrated by the inspectable official PathDeformPiece example. Do not infer that every current option was present in 18.0. [S84]
Vellum Brush	Since 18.5. Interactive settle/sculpt runs simulation internally; it is not an analytic static contact solution.
File Cache 2.0	Since 19.0. Static file or sequential simulation cache.
Vellum I/O 2.0 / RBD I/O 2.0	Since 19.5. Prior nodes existed; distinguish current versioned asset from first availability of the concept.
Current Guide Deform / Guide Utility	Current pages say Since 22.0. An older node named Guide Deform appears in the official HairOrient file; that does not establish availability of current Preserve Shape or interpolation controls in H21.
Vellum **Orientation Solve = Quasistatic**	H22 new bending/twist method. Exact location is Solver > Advanced > Motion > Orientation Solve. Not a control called Angular Integration. H22 release notes also add Angular Damping.

Three different uses of quasistatic must remain separate:

1. **Vellum Solver > Simulation > Simulation Type = Quasistatic** restarts from current inputs for each evaluated timeline frame and runs a chosen internal number of frames. The output has no normal cross-frame simulation continuity. This remains an internally integrated solve, not a direct geometric projection; the old HairOrient example already stores a quasistatic-frame count.
2. **H22 Orientation Solve = Quasistatic** changes the hair orientation/bend solve, ignoring angular velocity. SideFX recommends different stiffness ranges (including stretch around 100), and damping/stress behavior changes. Do not silently copy Dynamic-mode values into it.
3. **H22 Guide Deform > Preserve Shape** offers rod relaxation with stretch/bend/reference strengths and Lock Roots. No obstacle collision input/control is documented for this feature. It is not evidence that Guide Deform alone solves grass around a stone.

Sources: [S01], [S02], [S03], [S04], [S05], [S06], [S07], [S08], [S16], [S23], [S24], [S25].

tyFlow feature-history minimums

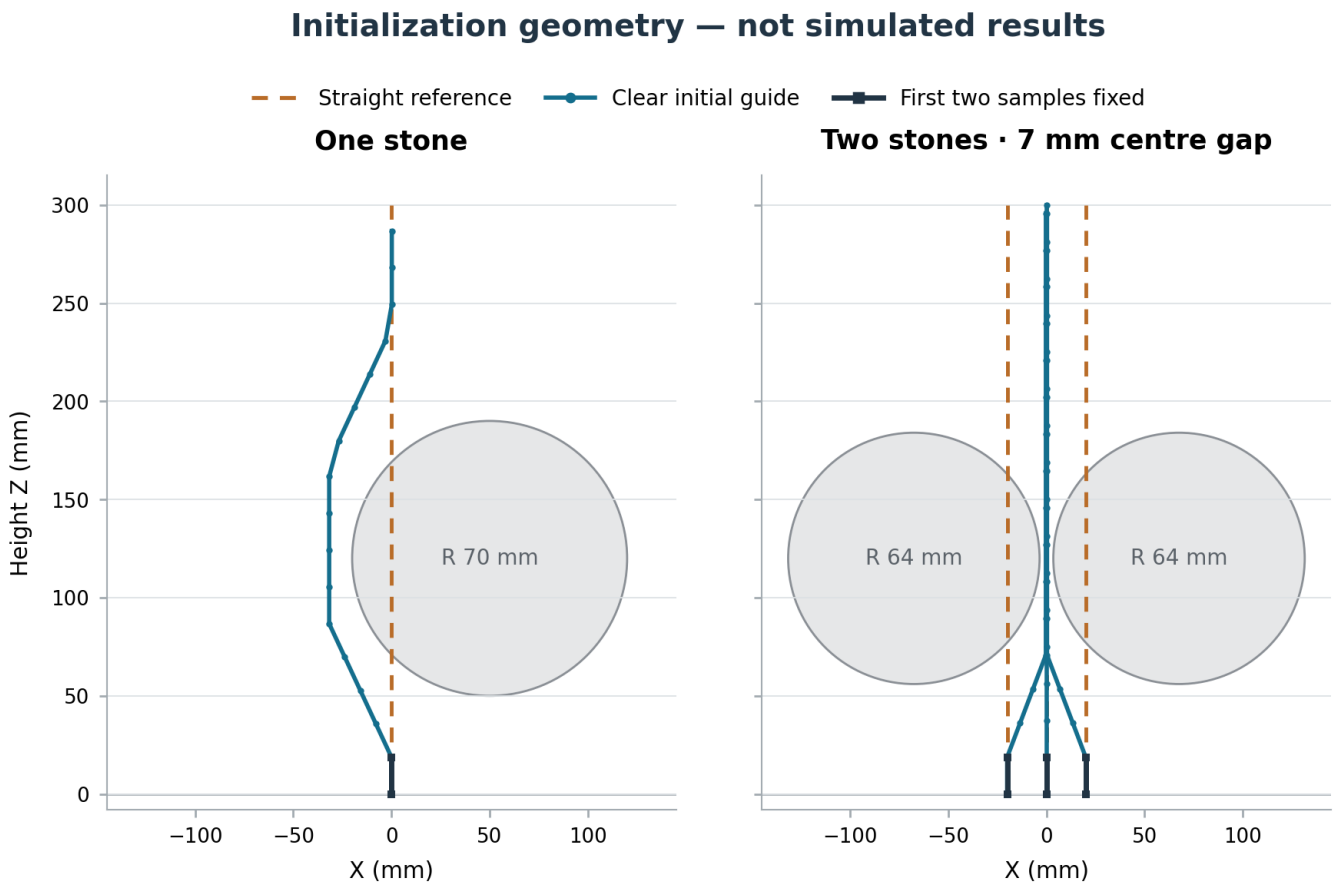
Useful minimum-history checks: CCCS was introduced in v0.16090; arbitrary mesh-driven tyParticleSkin in v0.16099 (20 December 2020); element-ID matching in v1.021 (8 June 2023); tySplineMesher arbitrary mesh deformation in v0.16055 (27 October 2019). This study targets v2.101 rather than promising all instructions work on those old versions. [S31]

SECTION 2

Common fixtures and reproduction rules

Use the same physical sizes and geometry roles in both applications. These are **proposed fixture specifications**, chosen to expose errors with a small scene. Production vegetation preparation remains a separate artist task.

Fixture	Geometry and dimensions	What the test must expose
Grass source	One 300 mm long, 3 mm wide segmented ribbon; 16 longitudinal segments. One ordered root-to-tip guide per blade. Keep an untouched source copy.	Root drift, base rotation, stretching, edge penetration and guide-to-mesh roll.
One stone	Closed smooth polygon sphere, 70 mm radius, centre at horizontal x = 50 mm and height = 120 mm. Root at the origin.	The straight blade intersects the stone above its root; an initialization decision is required.
Two stones	Two closed 64 mm radius spheres at x = -67.5 mm and +67.5 mm, height = 120 mm. Three blades rooted at x = -20, 0, +20 mm.	Simultaneous opposing contacts and per-blade association. The narrowest centre gap is 7 mm.
Pebbles	One closed 30 mm prototype, repeated 24 times in a separated 4 by 6 arrangement; later compare 20-40 mm variants. Terrain is a 1 m patch with broad irregularity up to 50 mm.	Shape-dependent rigid contact, sleeping, terrain concavity and preserved instance eligibility.
Leaves / curb	Leaf outline approximately 80 by 40 mm. Rigid version has a thin closed proxy, initially 0.5 mm thick; deforming version has an 8 by 4 segmented triangulated surface. Curb is 150 mm high.	Rigid leaning versus actual folding, thin-shape tolerances, collision silhouette and surface contact.



Shared input geometry only. Dashed lines are the straight reference; colored lines are the clear initial guides. Two-stone side guides leave the projection plane. These are not simulated results.

Coordinates and units: use metres and Y-up in Houdini; use centimetres and Z-up in Max. Convert a Houdini point `(x, y, z)` in metres to Max `(100x, -100z, 100y)` in centimetres. This is a proper rotation plus scale, preserving handedness. Thus both stone centres have the same physical interpretation. Geometry dimensions match; the native stiffness, damping and gravity controls are not assumed to have matching physical calibration across products.

Set 30 fps for the comparison. tyFlow recipes use frame 0 as their initialization frame. Houdini recipes identify their start explicitly; retain that setting in the scene record rather than silently shifting an example's initialization frame. Record the accepted frame and the actual number of evaluated steps. A nominal 90-frame window is a starting test interval, not a guarantee that an object has settled.

Initialization is part of the recipe

The straight-grass fixture intentionally intersects the final stone above the root. **Do not start a dynamic collision test embedded and call any resulting escape a reliable fit.** Choose and record one of these initialization branches:

1. **Direct static guide collision:** evaluate the final obstacle SDF with the static curve operation. This is the cleanest test of that operation's documented purpose; validate lengths, roots and the visible blade afterward.

2. **Clear initial curve, fixed obstacle:** author a collision-free starting curve. In a method with a separate straight/rest-shape target, let that target draw the curve toward the obstacle. The tyFlow Flat angle-target variant explicitly chooses a straight target; it does not preserve arbitrary source curvature.
3. **Conditioned simulation:** start the collider clear, move it gradually to its final transform, then hold it fixed and allow settling. This is a timeline-dependent loading history. It can produce a static final image, but is **not** a demonstration of a stone that was fixed throughout or a history-free static fitter.

For two stones, use both obstacles in the same solve. If conditioning is required, move them without initially intersecting the blade or each other. Repeat with the approach order exchanged to reveal history dependence. A gap narrower than the required blade envelope may prevent passage through that gap, but in three dimensions the blade might escape around the stones; gap width alone does not prove global infeasibility.

Attribute and preparation contract

Every grass blade needs an identified root, an ordered guide, enough mesh segments to bend, and a stable blade-to-guide association. A pivot is not enough to describe a disconnected clump. Keep UVs, material assignments and source IDs before capture. Verify that neighbouring blades do not capture each other's guides.

For Houdini, let the Vellum configuration nodes generate solver attributes and constraints. Inspect `P`, `orient`, pin state and collision thickness, and preserve point/primitive correspondence or the documented IDs. Rigid pieces need consistent `name` association between render geometry and proxies. For tyFlow, retain source/particle identity and explicit custom channels only where the recipe requires them; do not invent a Houdini attribute list for an operator that creates its own particle data.

For a rigid leaf, shading alpha is not a collision cutout. Prepare an appropriate geometric outline or accept the card's collision silhouette. For deforming leaves, an unsegmented plane cannot form a smooth fold. A high-resolution render leaf driven by a low-resolution proxy adds a binding task and a separate final-mesh collision check.

Counting manual work honestly

Counts are numbered artist tasks, not measured clicks or setup time. A task may contain several parameter entries. Section 12 reconciles the listed preparation, configuration, inspection and output tasks; shared output tasks are counted once.

The counts assume the small fixture, not an arbitrary imported production plant. Separating hundreds of blades, selecting roots, repairing scans, assigning per-element IDs or hand-authoring guides can dominate the job. Those operations are described as artist preparation and are never treated as automatic merely because a deformation node exists.

Section 12 contains the consolidated local acceptance record and validation checklist.

SECTION 3

Houdini: grass and one stone

Static fitting, a fixed-stone relaxation recipe, and a separately labeled conditioning alternative.

Shared guide-to-mesh decoding

Named Path Deform is the primary proposed route. It is supported by the operator help and the inspected official PathDeformPiece example. Connect the rest strip mesh to input 1 and the fitted/simulated guide to input 2. Enable matching geometry to curves using a piece attribute; use primitive `name` on both. For multiple blades, ensure every blade mesh has exactly its matching curve. One curve per blade avoids assigning a blade to a nearby neighbor in a tight clump. [S84]

For this fixture: set Forward Direction to Y; set the up direction to Z, which is not parallel to Y. Set the curve up control to a constant Z axis. Set position offset to curve U = 0; use the original geometry length (Fraction of Geo Length = 1), not automatic stretching to the whole curve. Confirm the geometry capture origin and centering put the authored root on the curve start. Set **Orient Using Point Attributes off**: the curve's Vellum `pscale` is a collision radius, and leaving all transform attributes enabled can wrongly scale the mesh. For the planar single-stone test this is a simple geometric bend frame. For the mild three-dimensional paths in H2C it remains a **proposed geometric frame convention** supported by Path Deform's up-axis controls, provided the curve tangent does not approach the chosen up axis. It does **not** transfer Vellum's material twist. Inspect for frame flips and compare the decoded strip against both obstacles. A physically twisting blade requires an explicitly authored and verified moving frame; material-frame collision fidelity is unvalidated here. Until that frame and final-mesh contact are qualified, H2C establishes a proposed **guide test**, not an accepted full-frame mesh result.

Precurved/wide assets: Wire Capture input 1 = rest mesh, input 2 = rest wire. Restrict each capture primitive entry to the matching blade mesh subgroup, or process one blade per loop; capture radius covers its width, not neighbors. Wire Deform inputs 1/2/3 = captured mesh/rest wire/deformed wire. Keep rest/deformed wire primitive counts and order stable. `Align Tangent And Attrib Normal` gives an explicit normal-aware frame; the official TwistWireDeform file authors `N` and a tangent and uses this mode. The appropriate conversion from a simulated material quaternion to a chosen normal basis must be authored and checked. Do not assume arbitrary `orient` will map to a blade's intended width axis.

Point Deform exception: it is appropriate with a genuine surface or volume cage, or low-resolution simulated leaf surface. Inputs = render mesh / rest cage / deformed cage, stable connectivity or supported ID mapping, and a piece attribute on both mesh and cage to restrict capture. It is not the recommended decoder for a single line per blade. The explicit warning in its help makes a claimed universal wire-to-mesh Point Deform recipe unsound.

Guide Deform distinction: H22 has a documented dense-groom workflow with sparse simulated guides: dense groom input 1, rest skin 2, animated skin 3, rest guides 4, simulated guides 5. The official Capybara shelf workflow is a grooming example. It does not automatically provide mesh skinning for arbitrary scattered grass assets. The H22 Solaris guide procedural similarly addresses groom data; use a separately documented mesh path for blades. Sources: [S06], [S07], [S10], [S11], [S12], [S13], [S78], [S79].

H1A. Conditioned Vellum Hair alternative

Documentation: [S02], [S03], [S04], [S14], [S15]. Mesh decoding: [S10], with inspected PathDeformPiece association. Output: [S23], [S24], [S25], [S26], [S27], [S28], [S29]. The scene assembly and numerical settings below are P.

Network: rest guides → root group → Vellum Constraints (Hair) → Vellum Constraints (Pin to Target) → Vellum Solver → optional cache → Path Deform with rest blade mesh → freeze/cache/export. The geometry and constraints branches run together from each Vellum Constraints node into the next. The collision stone connects to Solver input 3. There is no glue or attraction to the stone.

1. Import the rest strip and its 17-point guide, verify scale, root order, and matching primitive `name`.
2. Create a `roots` point group at the first vertex of each guide. The inspected HairOrient file uses `vertexprimindex(0, @vtxnum)==0`; its context is a Group Expression with point output. For hand-selected single-guide fixtures, selecting the first point is sufficient.
3. Create Vellum Constraints set to Hair on the entire guide. Use Compute Missing Orientation, Rest Orientation Frame = 1. Proposed trial: uniform point mass `1e-5`, uniform thickness/radius `pscale = 0.002`, Stretch Stiffness `1e6`, Bend Stiffness `1`, stretch/bend Damping Ratio `0.1`. Leave Pin Points empty here so root controls are independently authored downstream.
4. Create a second Vellum Constraints node set to Pin to Target for `roots`, feeding both geometry and constraints outputs from step 3. Use Pin Type **Stopped**, Orientation Pin Type **Same as Position**, Match Animation off for stationary roots. Leave mass/thickness unchanged on this node. This produces an explicit hard position and orientation test. For a flexible base variant, change only Orientation Pin Type to **Soft** and its Bend Stiffness to an independently chosen value, such as `10`; the position remains hard. The body bend setting remains at step 3.
5. Import a closed polygonal stone into the same coordinate space. Use initial stone centre (0.30, 0.12, 0) m and key a Transform from that pose at frame 1 to its final pose at frame 31, then hold it through frame 91. Verify the sweep itself does not cut through the pinned root. For a stone fixed from frame 1, use H1C below: it keeps original straight-rest constraints while initializing a different, clear curve. Merely rebuilding rest constraints from a pre-bent curve with gravity zero supplies no load that would bend it toward the stone.
6. Create Vellum Solver. Connect display geometry to input 1, its matching constraint geometry to input 2, the animated stone polygons to input 3. Use the full solver mode and Dynamic simulation. For H22 select Orientation Solve = Dynamic for this baseline.
7. Configure the trial: Start Frame 1, Substeps 5, Constraint Iterations 100, collision passes 10 and post-collision passes 5, gravity `(0,0,0)`, no wind. Zero gravity isolates the requested geometric deflection; enabling gravity is a different experiment. Leave other forces at their default neutral values. Set Advanced > Default Thickness to `0.0005` m for colliders without `pscale`; the guide already has `pscale=0.002` m. Their combined 2.5 mm envelope is a conservative proxy for a 3 mm-wide blade with clearance, not botanical thickness. Inspect incoming collider `pscale`, because it overrides Default Thickness.
8. Reset Simulation, return to frame 1, and run sequentially to 91. The stone is fixed after frame 31. Increase the hold if the guide is still moving; frame 91 is a trial endpoint, not a guarantee of equilibrium.
9. Inspect root position and outgoing-root direction separately, compare guide arc length, visualize thickness, and inspect failed external/self-collision markers. Measure the render strip against the stone after decoding as well; a nonpenetrating centreline is not alone a mesh collision guarantee. If several blades are simulated, decide whether inter-blade collision is part of the test; do not claim it if self-collision is disabled.

10. Create Path Deform as specified in the shared mesh-decoding instructions above, using the simulated curves and original mesh with matching `name`. Check root registration and frame orientation on one blade before expanding the batch.
11. Freeze the chosen result with Time Shift at the accepted frame. Save a single `.bgeo.sc` through File Cache with Time Dependent Cache off. Time Shift by itself is not a disk cache. For restartable simulation, save Vellum geometry **and** constraints with Vellum I/O, plus the collider animation separately.
12. Assign/render the deformed polygon geometry, and export the frozen mesh through Alembic or USD. Save the rest asset and guide association alongside it if later updates are required.

Action count: 12 after fixture modeling. The flexible-root variant changes one configuration in step 4. The optional per-piece Wire Capture route adds capture/frame preparation actions and should be reported separately, not hidden in this count.

Root semantics: `Stopped` sets the positional stop bit without destroying mass. Same as Position also stops orientation. Soft orientation adds a bend/twist target constraint and uses that node's bend parameters. A position-only pin (`Orientation Pin Type = None`) is a hinge; it does not preserve the outgoing tangent. Orientation pinning the tip does not control the incoming segment. These semantics are documented; the numerical behavior of the proposed material values is untested.

After moving the stone: turn off loading the previous cache, regenerate the clear-to-final motion or valid starting guide, reset/replay from frame 1, evaluate contact and roots again, then write a new cache version. Continuing an old simulation state after a teleport is a different history-dependent experiment. Do not silently carry its velocities and collision-disable state into a new static arrangement.

H1B. Documented static alternative: Guide Collide With VDB

Documentation: [S01], [S09]; mesh decoding [S10]. The native static operation is D, while the complete fixture and acceptance procedure are P.

This route has no timeline simulation. It is a reproducible **candidate static fit** with an explicit acceptance check, not a verified hard-root elastic workflow.

1. Import the same rest guide and named straight strip.
2. Import the stone in its final position; ensure a signed inside/outside representation is valid.
3. Create VDB from Polygons on the stone, with Distance VDB enabled and a proposed 0.001 m voxel size. Use a signed field, not an unsigned distance volume. If guides are deeply embedded, ensure the interior band covers them; Fill Interior is documented but can increase cost.
4. Create Guide Collide With VDB. Input 1 = guides; input 4 = obstacle distance VDB. Leave inputs 2 and 3 unconnected and **Collide With Skin off** for this external-obstacle-only test. The page explicitly documents input 4; it does not enumerate input labels 2 and 3. Their exact names are not independently verified here and are deliberately not invented.
5. Set Blend 1, Surface Offset 0.0025 m, Push Range 0 and Push Amount 0. Enable Curve Mask; set it to 0 at the root and through the first segment (normalized length 0-0.0625), rising to 1 by 0.15. This requests root-area protection; the documentation does not promise exact root invariance, orientation clamping, length preservation, or feasibility.
6. Evaluate at the current frame and compare root position, first-segment direction, guide arc length, and collisions to the undeformed reference. A mask is not an accepted substitute for these checks. If hard root invariance is required, a separate explicit restoration of the root point or first two points is possible as authored geometry processing, but may reintroduce stone

intersections and is **not** a documented native collision-plus-root solution. It is outside this minimal count.

7. Decode through named Path Deform with the settings in the shared mesh-decoding instructions above and validate the actual strip surface.
8. Save the accepted static result using File Cache with Time Dependent Cache off; cache the accepted guide as well if retaining editability.
9. Render/export the deformed mesh. Record the mask and VDB resolution with the result.

Action count: 9. Moving the stone recooks the distance field and static operator; no simulation reset is required, but a loaded disk cache must be disabled/replaced. Static fitting avoids a conditioning history but may fail the requested root/length/contact criteria. Do not present failed fits as a supported automatic result.

H1C. Fixed-from-start obstacle: separate straight rest constraints and clear initial geometry

Status: P, supported by D and I; not executed in Houdini. The documented two-stream solver architecture, rest attributes, hair orientation initialization, and the HairOrient example support this construction. HairOrient itself demonstrates deformed input geometry with the original constraint stream, but it does not demonstrate the proposed stone collision. The complete branch must therefore be locally qualified. Sources: [S02], [S03], [S14]; inspected HairOrient example; [S10] for decoding. [S77] [S80]

This branch supplies an actual driving load at zero gravity: the **original straight-rest bend constraints** act on a different, initially clear curve. Rebuilding the rest constraints from the clear curve would erase that load. A pre-bent rest curve with no external force should not be presented as self-bending around a stone.

1. Import the original straight 17-point guide and named strip mesh; keep the common point order and persistent blade identity.
2. Create `roots` as the first point of each guide.
3. Build Vellum Constraints set to Hair from the **straight** source, with H1A's body settings (`pscale=0.002`, point mass `1e-5`, Stretch `1e6`, Bend `1`, damping `0.1`, Compute Missing Orientation, Rest Orientation Frame 1). Leave its Pin Points empty.
4. Append Pin to Target for `roots`, keeping mass/thickness unchanged: Stopped position, Same as Position orientation, Match Animation on. Name its geometry output `REST_GUIDES_PINNED`; preserve its **constraint output unchanged** as `REST_CONSTRAINTS`. The soft-orientation variant changes this node's orientation mode and independent Bend setting, not the body-rest stream.
5. Import the one-stone clear initial coordinates in `common_grass_initialization.json` as a 17-point curve with precisely the same point order/IDs as the original. These points have the same segment lengths and the first two points remain vertical. They are a proposed fixture, not the result of a Houdini solve.
6. In a branch from `REST_GUIDES_PINNED`, copy only `P` from the clear curve by the unchanged point order/IDs. Preserve names, `roots`, mass, radius and pin state. Do not change point or primitive count.
7. Delete stale `orient` from this initial-geometry branch. Remove/zero any imported velocity or angular-velocity history if the source was previously simulated; this fixture starts from fresh source data with zero `v`/`w`.

8. Append a Vellum Constraints Hair node solely to initialize the now-missing orientations for the clear curve: Compute Missing Orientation on, Rest Orientation Frame 1, Mass and Thickness unchanged, no additional pins. **Discard this node's constraint output.** Its geometry output supplies correctly initialized hair attributes; it must not replace `REST_CONSTRAINTS` and make the clear curve the new rest shape.
9. Copy `P`, `orient` and the hard-pin state for the `roots` group from `REST_GUIDES_PINNED` back onto the initial-geometry branch. This explicitly preserves the planted root frame. Confirm the first edge is still consistent with that frame; inspect orientations along the remaining curve before solving. A missing or inconsistent orientation is a setup failure, not a collision problem.
10. Import the stone at its **final fixed** centre `(0.05,0.12,0)`, radius `0.07` m. Check the clear curve against the actual polygon collider and the combined 2.5 mm envelope. The JSON's sphere-distance arithmetic does not replace this check.
11. Connect Vellum Solver input 1 = initialized clear geometry, input 2 = original `REST_CONSTRAINTS`, input 3 = static stone. Use full mode. Set Advanced > Target Path to `REST_GUIDES_PINNED` so the root target is explicitly the original reference, not a changing initialization branch. Point correspondence must agree.
12. Choose one recorded evaluation mode. **Dynamic variant:** Simulation Type Dynamic, Start Frame 1, run 1-91. **Frame-local variant:** Simulation Type Quasistatic, Quasistatic Frames 90, evaluate the current input frame. For both use Substeps 5, Constraint Iterations 100, collision passes 10, post-collision passes 5, Default Thickness 0.0005 m, gravity zero, no wind, and H22 Orientation Solve Dynamic. Leave Clear Dynamics off during inspection so remaining motion can be examined.
13. Reset/recook and evaluate. Inspect root location/frame, length, collision flags, remaining motion and visible strip clearance. Increase the internal frame budget or dynamic duration only if the remaining issue is incomplete settling. A contact failure or orientation error is not repaired by calling the result quasistatic.
14. Decode the named strip with Path Deform, following the shared mesh-decoding instructions above, and check its actual surface against the stone.
15. Save the accepted result through File Cache as a static output; the Dynamic variant can first use Time Shift at the accepted frame, while the frame-local variant caches the accepted current-frame cook. Preserve the separate rest/initial geometry branches and constraints for updates. Clear Dynamics can be enabled on the published static output after motion acceptance if desired.
16. Render/export the frozen mesh with the ordinary output checks.

Action count: 16 for either evaluation mode. This is more preparation than static Guide Collide because rest shape, initial shape and material orientation must be kept distinct. Quasistatic hides timeline traversal from the artist, but it still internally runs a simulation for the specified number of frames. It does not guarantee a unique or converged solution.

After moving the stone: recook the actual collider, select or author a clear initial curve for the new arrangement, retain the original straight-rest target, recompute initial orientations and reset/recook. If the new obstacle overlaps the pinned root or no valid allowed shape is found, flag failure. Do not use the previous solved geometry as a new rest state silently. No dependence on the previous result is required by the frame-local mode when these inputs are fixed; the choice of clear starting geometry can still affect the final equilibrium.

SECTION 4

Houdini: grass between two stones

H2A. Conditioned Vellum Hair with simultaneous contact

Documentation: [S02], [S03], [S04], [S14], [S15], [S18]; mesh decoding [S10]. The complete two-stone scene is P.

This is not a pair of sequential single-obstacle pushes. Both obstacles must participate in the same solve, and their final gap must accommodate the modeled collision envelope. A fixed root can make a geometrically narrow or closed gap infeasible regardless of iterations.

1. Import rest guide and strip; validate scale, root order, and matching names.
2. Create the `roots` group as in H1A.
3. Create the Hair constraint node with the H1A mass, thickness, stretch, bend, and orientation initialization.
4. Create the independent Pin to Target node, hard positional root and hard orientation for the baseline; expose soft orientation separately as the variant.
5. Import the left stone at (-0.30, 0.12, 0) m and author its clear-to-final conditioning transform (frames 1-31), holding its final pose thereafter.
6. Import the right stone at (0.30, 0.12, 0) m and author its clear-to-final transform on the same frames. Choose initial poses that do not overlap the grass envelope or its pinned root.
7. Merge both polygon collision branches into one stream. Inspect the **final** clearance before solving. The 2 mm guide radius plus 0.5 mm external-collider thickness requires at least 5 mm centreline-channel width for a simple parallel slot; curved surfaces, root location, mesh width, and desired margins can require more. This is a geometric lower-bound inference, not a solver guarantee.
8. Create the full Vellum Solver with geometry input 1, constraints input 2, and the merged two-stone geometry input 3.
9. Configure Dynamic mode and the same solver values as H1A step 7; use H22 Orientation Solve = Dynamic for the baseline.
10. Reset, return to frame 1, and run to 91 or a longer accepted settling frame. Both stones are fixed from frame 31.
11. Validate both contacts together, roots, guide length, and collision-failure markers. If symmetry permits two bend directions, author a tiny initial pre-bend toward the desired side and record that choice. Increasing iterations does not choose an artistic side or open an impossible channel.
12. Decode the named meshes with Path Deform and check contacts on both sides of the strip.
13. Freeze and cache the accepted frame; preserve solver state/colliders separately if restart is required.
14. Render/export the frozen mesh, preserving the reference asset and names for later updates.

Action count: 14. The distinct preparation burden is two obstacle histories, their merge, and a two-sided feasibility/contact check. Starting from a complete H1A network instead is two additional graph actions (import/configure the second obstacle and merge it), with existing inspection updated.

Moving either stone: invalidate the joint solution, regenerate valid conditioning for both obstacles, reset/replay, then validate and replace the cache. A local change can affect the entire constrained guide; no locality guarantee is documented. If the channel has become too small, report the conflict instead of pushing the root or stretching the blade silently.

H2B. Static candidate with two VDB obstacles

Perform H1B with two stone imports, a VDB from Polygons on each, and a Merge of both fields into Guide Collide's input 4. Its help refers to VDBs in the plural. Disable skin collision. Inspect both contacts after one joint evaluation; do not claim that separately running one push after another converges. This adds three actions to H1B (second stone, its VDB, field merge): **12 actions**. A coarse voxel grid can erase a thin gap. No hard-root guarantee or infeasibility detector is documented, so this remains an explicitly checked static candidate.

H2C. Both stones fixed from the beginning, frame-local or dynamic relaxation

Use the complete H1C branch, with the three straight guides rooted at x=-0.02, 0, +0.02 m and the matching three clear initial curves from `common_grass_initialization.json`. The original straight-guide constraints remain the material rest target. Import both final stones at x=±0.0675 m, y=0.12 m, radius 0.064 m; merge their polygon geometry into Solver input 3. The two additional actions are the second stone import and the collider Merge, for **18 actions** total. Both obstacles participate in every internal step. Use the same per-blade names for decoding, and self-collision if inter-blade contact is part of the trial.

The common coordinates deliberately keep separate guides and a clear first segment. Their ideal-sphere preflight has more than 2.5 mm initial centreline-to-sphere clearance, but polygonal tessellation and actual strip clearance still need checking. The 7 mm centre slot has only a small margin over the 5 mm effective guide envelope. A side blade may pass around the spheres in three dimensions; report the accepted route and root frame rather than assuming a unique 2D answer. If the requested route is specifically through the centre gap, that is an additional artist constraint, not something implied merely by selecting two stones. The constant-up Path Deform option in Section 3 is a geometric decoder and omits Vellum twist; qualify its frame and both-side strip contact before calling the mesh result accepted. Without that check, retain H2C as a guide-level test. Sources: [S02], [S03], [S10], [S14], [S15] and the same inspected HairOrient architecture; complete two-stone result remains P. [S77] [S80]

SECTION 5

Houdini: rigid pebbles

Documentation: [S20], [S21], [S22], [S27]; output [S23], [S25], [S26], [S27], [S28], [S29]. RBD Bullet Solver's official OTL was unpacked for inspection, but it is not a demonstrated pebble fixture. The scene below is P.

Network: source prototypes + spawn points → Copy to Points (Pack and Instance) → unique piece names → RBD Configure → RBD Bullet Solver → freeze/cache → render/export. Terrain enters Solver input 4. Constraints input 2 is empty; optional collision proxies enter input 3 with the same piece names as render geometry.

1. Import a small set of closed pebble prototypes (one 0.03 m prototype, with 0.02-0.04 m variants as a later test) and preserve source IDs.
2. Author 24 spawn points/orientations in a separated 4-by-6 arrangement above the terrain with no initial overlap. Use deliberate reproducible positions/seeds, not an overlapping pile that is expected to depenetrate.
3. Use Copy to Points with **Pack and Instance** enabled, with source selection by piece attribute if several prototypes are used.
4. Assign a unique primitive `name` to every packed instance while retaining the source/prototype identity separately. Avoid conflicting point and primitive names; packed fragments have different name conventions from ordinary packed primitives.
5. Import a 1 m terrain patch with broad irregularity up to 0.05 m as polygonal collision geometry. Keep its actual relief; an infinite plane is not an adequate substitute.
6. Apply RBD Configure to the pebbles: active on, animated/deforming off, Convex Hull collision shape, initial Sleeping off, Enable Sleeping on. Trial settings: Bounce 0, Friction 0.6, Collision Padding 0.0005 m with matching shrink enabled, Add Density on, User Density 2000 kg/m³, variation 0, Unit Mass 1 kg and Unit Length 1 m (proposed fixture values). Convex hull is suitable for approximately convex pebbles; if cavities matter, the collision model must change. Optional proxy geometry must have matching names and transforms.
7. Create RBD Bullet Solver: render geometry input 1, no constraints input 2, optional proxies input 3, terrain input 4, no guiding input 5.
8. Configure external collision as **Static**, **Concave** for irregular terrain where a hull would bridge valleys; external Bounce 0 and Friction 0.6. This is the solver's Collision tab; body representation is set on the RBD Configure branch, not accidentally changed by the external-collider tab.
9. Set Start Frame 1, gravity `(0,-9.81,0)`, proposed Bullet substeps 10 and constraint iterations 50; reset and run sequentially to a trial frame 91. Enable the appropriate collision-shape and sleeping visualizers to assess the result. If tiny shapes behave poorly, the documented Bullet World Scale is a specific remedy, with dependent attributes scaled by the solver.
10. Inspect actual simulation shapes against render meshes, interpenetration, hovering, and remaining motion. A sleeping/low-velocity state is required for an accepted settled layout; frame number alone is insufficient.
11. Freeze the chosen frame with Time Shift and save a static File Cache. For efficient animation storage, RBD I/O can store simulation points/transforms with a separately cached high-resolution rest geometry and transform it on load.

12. Render packed prototypes; export through USD native instances/PointInstancer or Alembic with instancing enabled where applicable. Verify the receiving renderer/importer preserves the desired reuse and materials.

Action count: 12, plus explicit proxy-authoring steps if render meshes are too detailed. This is timeline simulation; no direct static RBD settling operator is established by these sources. The subsequent frozen frame is static output.

Failure remedies: hulls bridge concave terrain; choose concave triangles for the static terrain. Large padding creates hovering; scale margin and enable matching shrink. Very small/thin bodies need suitable scale and stable collision thickness; World Scale and substeps address specific numerical issues. Initial overlap can eject objects; fix starts, or deliberately evaluate the documented overlap/split-impulse options rather than claiming a general overlap solver. Names must match render and proxy pieces or transforms can go to the wrong geometry. More substeps improve collision resolution but do not recover holes absent from a convex collision shape. Sources: [S20], [S21], [S22], [S23], [S24], [S25], [S26], [S27], [S28], [S29].

SECTION 6

Houdini: fallen leaves at a curb

H4R: Rigid leaves

Documentation: [S20], [S21], [S22], [S27]; output [S23], [S25], [S26], [S27], [S28], [S29]. Thin leaf proxy and leaning-pose choices are P.

Rigid leaves retain their authored shape. A flat or curled render leaf may rotate/translate and stack, but it cannot bend over the curb. Geometry sharing survives because each leaf is still a transformed prototype. Alpha transparency in a shader is not a polygon collision silhouette; author a suitable silhouette/proxy.

1. Import the 0.08-by-0.04 m render leaf prototype with UVs; use the flat common fixture first, and treat authored curl as a later variant.
2. Prepare a closed thin collision proxy with matching silhouette, proposed 0.0005 m total thickness. A bare zero-thickness card is not this recipe. If the leaf is concave/curled, acknowledge that a single convex hull fills its cavities; select a more faithful documented shape only when those cavities matter.
3. Use the common clear leaning pose: centre `(-0.0199,0.04125,0)` m, length direction `(cos(65°),sin(65°),0)`, width direction `(0,0,-1)`. Endpoints are approximately `(-0.0368,0.0050,0)` and `(-0.0030,0.0775,0)`. Place the curb face at x=0, ground at y=0 and curb top at y=0.15 m, with the leaf on the exterior negative-X side. This produces a gravity-driven lean/contact test; duplicate three more leaves along Z with spacing greater than 0.04 m only after the single leaf works.
4. Copy/pack/instance render leaves and matching proxies with the same placements.
5. Assign matching unique piece names to both branches, keeping prototype IDs separate.
6. Import the ground and curb meshes.
7. Merge ground and curb as external polygon collision geometry.
8. Configure the leaf proxies with RBD Configure: active on, animated/deforming off, initial Sleeping off and Enable Sleeping on, Convex Hull for the simple closed prototype, Bounce 0, Friction 0.6, proposed padding 0.0001 m with shrink. Use Add Density on, User Density 500 kg/m³, variation 0, Unit Mass 1 kg and Unit Length 1 m for this proposed proxy test. Retain the ordinary geometry-derived inertia. This chosen density is not measured leaf tissue data; changing proxy thickness changes volume-derived mass.
9. Connect the RBD Bullet Solver: render input 1, empty constraints input 2, proxy input 3, merged ground/curb input 4. Use Static/Concave external geometry. Set trial Bullet substeps 10, gravity -9.81 Y, Start Frame 1. Initial `v` and `w` are zero; use the common leaning pose so this remains paired with the gravity-only tyFlow test.
10. Reset and play forward to an accepted settled state, checking the closed proxy, curb contact, and sleep/motion status.
11. Freeze/cache the settled transforms or packed geometry.
12. Render the detailed leaves and export instances/transforms using the same choices as H3.

Action count: 12, with collision-proxy modeling counted as one preparation action whose actual modeling time is not predicted. Reset/replay after moving the curb or terrain; retain sharing as in H3. Thin margins larger than the proxy thickness, excessive spin, bad collision silhouettes, and

initially intersecting leaves are material failure risks. A result that needs the leaf to drape cannot be fixed merely by tuning a rigid solver. Sources: [S20], [S21], [S22], [S23], [S24], [S25], [S26], [S27], [S28], [S29].

H4D: Deforming leaves

Documentation: [S02], [S03], [S14], [S15], [S16], [S17], [S18], [S19]; optional surface-cage decoder [S13]; output [S23], [S24], [S26], [S28], [S29]. The leaf-specific setup is P.

Network: tessellated leaf surfaces → Vellum Configure Cloth (or Vellum Constraints set to Cloth) → Vellum Solver → optional Post-Process / surface-cage Point Deform → freeze/cache → render/export. No root pins or attachments; these are fallen leaves. Ground/curb polygons enter input 3.

1. Import the consistently wound 0.08-by-0.04 m leaf surface with 8-by-4 segments, triangulated, and UVs. Use the same flat outline as the rigid baseline; authored rest curl is a later variant.
2. Place the leaf in the exact H4R step 3 leaning pose, then duplicate three more leaves along Z with spacing greater than 0.04 m after the single-leaf test works. Assign unique piece names and avoid duplicate point IDs when merging.
3. Import ground and curb polygon collision geometry.
4. Merge the ground and curb collision branches in a stable topology stream.
5. Add Vellum Configure Cloth / Constraints Cloth: no pins, stretch and bend enabled, rest shape taken from the authored leaf. Proposed uniform radius `pscale=0.0005` m, Stretch Stiffness `1e6`, Bend Stiffness `0.1`, damping ratios 0.1. Use Mass = Set Uniform, point mass 1e-5 for this proposed fixed-topology fixture. Record total leaf mass; changing subdivision changes total mass in this mode, so use a deliberately recalibrated mass/density mode for a refinement comparison. These values are a trial and do not establish dry/wet leaf material accuracy. Inspect thickness relative to edge length.
6. Add full Vellum Solver: leaf geometry input 1, matching constraints input 2, merged collision geometry input 3. Dynamic mode; Start Frame 1, Substeps 5, Constraint Iterations 100, collision passes 10, post-collision passes 5, gravity `(0,-9.81,0)`. Enable self-collision for a multi-leaf pile. Keep wind off for comparability. Set initial `v` and `w` to zero; contact is driven by the common leaning pose and gravity, with no unpaired horizontal push. Set Advanced > Default Thickness to `0.0005` m for colliders without `pscale`.
7. Reset and run sequentially to a trial frame 91, extending until the accepted settled state. Do not assume the material has equilibrated merely because it reached that frame.
8. Inspect penetration, failed collision flags, stretch, bend, and retained motion. Increase bend stiffness to represent a stiffer leaf, or change to the rigid workflow if deformation is not wanted. More substeps/iterations improve constraint convergence; an impossibly pinched sheet between overlapping obstacles has no valid contact solution.
9. Add Vellum Post-Process only for necessary welding/normals/display conversion. Leave smoothing/subdivision off for the baseline. If either is added, validate again: SideFX documents that post-processing can introduce intersections and Detangle has limits. For a detailed render leaf, Point Deform from this **surface** cage is a separate valid option with matching per-leaf IDs; it is not the rejected single-wire capture.
10. Freeze the accepted frame. File Cache stores the final geometry; Vellum I/O with geometry and constraints supports a fuller restart state. The I/O node does **not** cache its third collision input, which must be saved separately.
11. Render the deformed surfaces with the original UV/material data; add render thickness deliberately if needed and check contact again if geometry is expanded.

12. Export static or animated deformed geometry to Alembic/USD. Each distinct leaf shape is unique geometry; do not promise the original rigid prototype sharing merely because the results are packed afterward.

Action count: 12 after preparing the surface asset. A high-resolution surface-cage transfer is additional setup, not included. The analogous single-frame Vellum Quasistatic mode can internally settle each input frame, or Vellum Brush can interactively settle, but both run simulation. Their convenience does not establish an analytic contact fit or automatic collision-free initial overlap repair.

Documented cloth remedies: thin geometry needs deliberate thickness; a newly created cloth configuration can reset thickness to 0.01, too large for small leaves. Higher bend changes the degree of wrinkling. Five substeps is an official starting suggestion for cloth. Duplicate IDs and inconsistent winding cause trouble; clean IDs and use appropriate winding repair. Very high damping can impede convergence. Collision-pass changes should be balanced with constraint iterations; excessive post-collision passes can make protrusions. Fix invalid initial geometry instead of concealing disabled collision markers. Sources: [S02], [S03], [S13], [S14], [S15], [S17], [S18], [S19], [S23], [S24], [S25], [S26], [S28], [S29].

SECTION 7

tyFlow: grass and one stone

All complete assemblies in this section are **P**. The fixed-stone guide route C and conditioned direct-surface route A have different root-frame and initialization boundaries. The static correction tools at the end are narrower than either complete workflow.

Shared tyFlow grass starting controls (P): History-dependent evaluation, Time step 1/4 Frame, Time scale 1, Manual reset, Particle Bind Solver Steps 40, Stepped force integration on and Deterministic partitioning on; CCCS Repulse steps 5. Record the installed v2.101 Impulse/IZ budgets, contact friction and any other inherited defaults before the run. Slow: Velocity X/Y/Z = 5 in the native UI amount controls, Variation 0, Only Affect = All values; Spin X/Y/Z = 0; Slow on subframes off; property-relative and surface-proximity influence off. These are proposed starting choices, not measured material coefficients or guaranteed convergence settings. For the direct ribbon choose stretch/shear 1, bend 0.2, volume 0, Constrained stiffness, damping 0.05, base/self thickness 0.05 cm and Mass per-cloth 1.0; disable tearing. For the guide, use the 0.2 cm binding radius and 0.05 cm collider envelope specified below. [S52] [S53] [S54] [S58] [S46]

Feature responsibilities

Feature	Documented responsibility	Important boundary
Particle Bind	Creates distance-like bindings, including neighboring siblings; the global bind solver processes them later. CUDA collisions can include the binding segments themselves.	A Collision operator alone is not proof of continuous collision along every guide segment. Enable the binding CUDA collision path.
Angle Bind	Creates constraints between adjacent existing bindings, with angular and area stiffness; can use the current angle or Flat (180 degrees) as target.	No documented independent twist degree of freedom or torsional grass-blade frame follows from its name.
Particle Switch	Deactivated particles have infinite mass to the bind solver.	This does not establish that arbitrary later position/force operators cannot move them. Filter those operators or leave them out of the minimal fixture.
Object Bind	Independently follows object/surface position, rotation or scale; may use springs.	Rotation-following on an isolated root particle does not by itself impose the tangent or twist of a positional chain.
Hair Bind	Attaches particles to **Ornatrix** hair, after Position Hair has recorded the required strand transform channels.	It is not tyFlow's generic hair solver and is not a substitute for Particle Bind/Cloth Bind.
tySplineMesher Mesh mode	Deforms an arbitrary input mesh along a spline.	Its generated frame and twist controls are a mesh-deformation choice, not physical rod torsion.
tyParticleSkin	Deforms a mesh from particle transforms or an input mesh's vertex transforms, using stored reference weights.	It does not create dynamics, discover a root, guarantee per-blade separation, or make an arbitrary sparse guide carry a stable blade frame.

Sources: Particle Bind [S36], Angle Bind [S37], Particle Switch [S38], Object Bind [S39], Rotation [S40], Hair Bind [S41], tySplineMesher [S49], tyParticleSkin [S50]. The Rotation documentation explicitly says the core binding solver does not update particle rotations. There are rotation reconstruction modes, but no documented claim of a fully framed centerline rod should be inferred.

Root position, tangent and frame

These must be separate controls in a Cyrus implementation.

- **Position only:** deactivate the first guide particle. Its coordinate remains fixed in the bind solve, but the first segment can pivot around it.
- **Hard tangent:** deactivate the first two guide samples. Their relative positions fix the base segment's direction. This is a geometric consequence, not a claim about a hidden rotational solver.
- **Twist/full base frame:** two collinear samples still permit rotation of a blade cross-section about their common axis. A noncollinear patch of at least three fixed ribbon vertices gives a full geometric base frame. Cloth Bind can deactivate a base patch by face material ID.
- **Orientation stiffness above a clamp:** pinning a base patch is a hard constraint. The flexible transition above it comes from bend/shear stiffness and optionally a stiffness map. Do not describe that map as changing the strength of the fixed pins themselves.

For a rooted plant with a local source frame, store/prepare the source root position and at least a tangent plus cross-section direction. A root height threshold is adequate only for the controlled flat-ground fixture below; it is not a general arbitrary-terrain root discovery algorithm.

C: Fixed-from-start guide chain

Status: proposed complete assembly of documented rope/binding/spline features. This is an alternative to A for cheaper guide collision and mesh deformation; it is not demonstrated here as a framed grass rod.

A reproducible collision-free fixture

Use the exact shared input in **`common_grass_initialization.json`**. It supplies an untouched straight source and a clear open guide, each **30 cm total polyline length with 17 samples and 16 equal 1.875 cm segments**. In Max centimetres, start at `(0,0,0)`; the first segment is vertical. Successive segment directions, measured as degrees from +Z toward +X in the XZ plane, are `[0,-25,-25,-25,-25,0,0,0,0,15,25,25,25,10,0,0]`. The fixed closed polygon sphere has centre `(5,0,12)` cm and radius 7 cm. Preserve these samples rather than resampling them inconsistently.

Executed geometry arithmetic only: summing the segment lengths gives 30 cm; the minimum centreline distance from the ideal mathematical sphere's surface is 1.169636963 cm. This is a line-segment/sphere calculation, not a tyFlow, polygon collision, rendering or export test. Inspect the actual triangulated sphere and full decoded blade in the application. The JSON contains both Max and Houdini coordinates, construction directions and calculation formula. It does not claim a converged simulation.

Unlike a current-angle-rest chain that would simply retain the pre-bent shape, **Angle Bind > Initial Angle > Flat** supplies a 180-degree angular target. The pre-bent chain therefore tries to straighten while both roots and stone remain fixed. Its segment rest lengths come from the initialized polyline. This is dynamic relaxation toward a deliberately chosen straight target, not recovery of an arbitrary imported rest curvature.

Put a root-threshold helper at **Z=2 cm**, between the second and third guide samples. In this fixture it marks exactly the first two samples. For position-only anchoring, lower it to **Z=0.1 cm** so only the first sample is marked. Do not use this height trick on general terrain or on loops that return below the threshold; prepare explicit root metadata there.

One-event operator order

1. **Birth Spline:** pick the source spline; start=end=0; birth at the prepared spline knots, once. The official rope FAQ documents sibling binding of particles born on splines. For multiple guides, enable **Save birth index** into the chosen channel `guide_id`; this documented 0-based value distinguishes source objects/subsplines. It is not an assumed universal plant attribute.
2. **Custom Properties:** on event entry, SET an absolute float value 0 in a chosen channel named here `rootmask`; leave source/target cross-particle sharing off.
3. **Object Test:** on event entry, test **Below Object (World-Z)** against the root-threshold helper. In Test Action enable **Increment custom float**, channel `rootmask`; disable **Send Out**. The channel becomes positive for roots while all particles stay in the same event.
4. **Particle Switch:** filter to custom float `rootmask > 0`; **De-activate bindings**. This makes the selected roots infinite-mass bind-solver anchors. The rest retain normal activation.
5. **Particle Bind:** on entry, bind to **siblings**, **Proximity = 1**; turn off proximity-by-distance or other bind generation modes that might cross-link neighboring blades. Do not close first to last for an open blade. Use a distinct binding ID, e.g. 10; stretch stiffness 1, no breakage. Enable **CUDA collision solver** and **Object collisions**. For the 0.3 cm wide blade, use a proposed **0.2 cm binding radius**: half-width 0.15 cm plus 0.05 cm margin. This is a conservative envelope, not a thin ribbon collision shape.
6. **Angle Bind:** after Particle Bind, use the filter for binding ID 10 and a distinct new binding ID, e.g. 20. Use **Initial Angle = Flat**, angular stiffness 0.1 for the proposed starting run (test 0.05-0.2 separately), area stiffness 0 initially. Keep Invertible at its documented/default safe setting; the docs warn disabling it may cause instability. The exact angular value is not a physical grass modulus.
7. **Slow:** use the explicit shared Slow starting settings at the beginning of this section. In this minimal fixture add no gravity or force: the flat-angle target provides the motion. This avoids assuming infinite mass disables generic forces.
8. **Collision:** add the fixed stone to the same event; low/zero bounce and intentional friction; proposed **Absolute radius = 0.05 cm** gives collider thickness to CCCS. Other Collision radius choices do not set the CCCS object thickness. Add both stones here for the two-stone variant. Set Collision Evaluation timing = After secondary solvers. Keep ground visual-only for this force-free fixture so roots at Z=0 are not embedded in a positive-thickness ground collider.
9. **Spline Paths:** mode **Particle bindings**; filter **binding ID 10 only**, so auxiliary angular bindings do not create unwanted render splines. Pick/create a tySplines output object and set a stable export ID if caching splines. For multiple guides, enable **Clustering**, channel `guide_id`, so any later spatial knot welding can be constrained to matching guide IDs. Current v2.100+ UI accepts a bind-ID filter list rather than only the older single-ID spinner. [S42] [S43] [S44] [S45] [S36] [S37] [S38] [S47] [S58] [S72]

The global Particle Bind Solver executes the constraints after the event's operators. Its position corrections do not automatically calculate particle rotation. When debugging, display particle IDs/sibling chains, original segment bindings, root mask and collision radius before making a dense clump.

Guide-to-visible-mesh chain

On the **tySplines** object enable **Weld bindings** and **Sort bindings** for contiguous ordered guide paths. Leave spatial **Weld knots** and **Close loops** off for the baseline; do not accidentally join crossing blades. If spatial knot welding becomes necessary, enable **Cluster knots** using the transferred `guide_id` data and choose a small tolerance. Start with linear interpolation and no simplification, so the visible path matches the solved segments. Smooth interpolation can bend between simulated samples and requires renewed collision inspection.

Add **tySplineMesher**, mode **Mesh**, and select the segmented 30 cm by 0.3 cm straight source blade. Prepare its root pivot and longitudinal axis consistently (e.g. +Z). Set direction axis Z, **Center pivot off**, and use **No repetition**. Use **Spline length = 100%** for this equal-length fixture; **Mesh bounds = 100%** is the documented alternative length convention. Disable Assign UVs when original blade UVs should survive, and inspect material IDs. Verify that the decoded root maps to the first/root knot, using the source axis/flip convention deliberately; do not accept a reversed or translated root. Use the spline-derived orientation as an initial deformation convention; arbitrary spline-frame twisting is not solved by two fixed guide points. Its twist controls are artistic deformation parameters. [S48] [S49] [S59]

A source blade with no longitudinal vertices cannot acquire a smooth bend merely because its centerline has many points. Conversely, a wide rendered blade can intersect a stone while its centerline clears: the conservative guide radius must cover the width, or use the direct ribbon solver. Dense two-stone gaps can be falsely blocked by a conservative circular envelope that is wider than an edge-on ribbon.

Artist work count

The one-guide route has **20 numbered artist tasks**:

1. Set units, fps and frame range.
2. Prepare/retain the 30 cm by 0.3 cm segmented render blade.
3. Create the 17-sample clear ordered guide from the fixture.
4. Prepare the final closed stone mesh.
5. Create the root-threshold helper at Z=2 cm.
6. Create an empty flow and configure Birth Spline.
7. Configure Custom Properties for `rootmask` initialization.
8. Configure Object Test and its increment action.
9. Configure filtered Particle Switch.
10. Configure sibling Particle Bind and segment collisions.
11. Configure Angle Bind and the Flat target.
12. Configure Slow.
13. Configure Collision and its source list/envelope.
14. Configure main timestep, bind Steps and CCCS quality.
15. Configure Spline Paths and select/create its tySplines target.
16. Configure tySplines welding/sorting/interpolation.
17. Configure tySplineMesher, source axis/pivot, length mapping and UVs.
18. Advance, inspect roots/guide/render mesh, and select an accepted frame or report failure.

19. Write the selected cache and hold/export the accepted pose.
20. Reopen/render/export-check the visible result.

A: Conditioned direct ribbon

Status: proposed documentation-derived scene assembly. **Supported building blocks:** Birth Objects, Cloth Bind, Collision, Particle Bind Solver, Slow and Particle Export. **Evaluation:** timeline simulation; no documented single-frame equilibrium guarantee. For the common straight-blade fixture this is explicitly a conditioned loading history, because the source blade intersects the final sphere. It does not demonstrate a stone fixed throughout the solve. Recipe C supplies that separate fixed-from-start test.

Artist input and preparation

Use the shared fixture: Max centimetres, Z up, 30 fps, frames 0-90. Prepare a **30 cm long, 0.3 cm wide ribbon with 16 longitudinal segments**, at least two vertices across its width, and an untouched source copy. This resolution is a proposed test specification, not a vendor prescription. Use clean, stable topology; retain UVs and materials. Make the first base quad/triangle patch a dedicated material ID, e.g. ID 2, with the rest ID 1. Assign both IDs the same visible material if the pinning ID should not affect appearance. A fixed base patch should contain at least three noncollinear vertices; a single transverse edge only supplies two fixed points.

The final stone is a closed polygon sphere of **7 cm radius, final centre (5,0,12) cm**. Its triangle mesh, not a PhysX convex hull proxy, is the collision representation for this CCCS cloth workflow. Keep ground visual-only in the minimal root test; a positive-thickness pinned root at Z=0 must not start embedded in a ground collision surface. If adding ground collisions, arrange a clear anchor envelope or an explicit suitable exclusion and inspect it. Confirm the initial ribbon, including collision thickness, is outside every collider.

Initialize the stone at **(30,0,12) cm at frame 0**, animate it smoothly to **(5,0,12) cm at frame 30**, then keep that transform fixed through frame 90. Start with no gravity, wind or noise; the moving stone supplies the load. Advance from frame 0. Label it **animated-obstacle conditioning followed by fixed-obstacle settling**. It is not static fitting, and 90 frames is only the proposed observation window. Avoid direct teleportation into the cloth. A larger timestep or sharp keyframe velocity change may require slower motion/refinement.

A separate, all-fixed direct-ribbon gravity experiment can be authored with an already-clear leaning blade beside a lower stone, but that changes the common initial/source fixture. No documented separation of straight rest constraints from a pre-bent current Cloth Bind ribbon was established here, so this report does not claim that exact full-frame/fixed-from-start version is solved merely by changing one option.

Operator/event order

Use one event so the cloth's Collision object list is unambiguous:

1. **Birth Objects:** pick the blade mesh; create at start frame 0 once; inherit its mesh. For a stationary rest input, disable continuous object position/rotation/scale inheritance so the source does not overwrite simulation. A Shape operator is unnecessary when the birth already supplies the mesh.

2. **Cloth Bind - Shape to cloth:** convert the incoming blade particle to a dynamic cloth mesh. Choose **De-activate by material ID**, target ID 2; choose **Include border vertices** deliberately so the full intended base patch remains fixed. Inspect its activation display.
3. **Slow:** damp velocity; keep it above any optional force-adding operators. Use the explicit shared Slow starting settings given at the beginning of this section. It controls settling speed, not rest curvature or root stiffness. There is no Force operator in this conditioning baseline.
4. **Collision:** add the animated-then-held stone; use its actual mesh. Keep collision response at zero bounce for settling; tune friction intentionally. Set its evaluation to **After secondary solvers** when using its normal collision response, as documented. For CCCS, its **Absolute radius** alone supplies the collider's thickness; shape/scale radius settings are not substituted into CCCS. The proposed collider envelope is 0.05 cm.
5. In Cloth Bind enable **Generate Cloth Mesh**: this is the actual visible cloth surface. A generic Mesh operator for particle shapes is not a replacement for it and is not added to this baseline. Avoid render-only different input subdivision that would change the simulation between viewport and render.
6. **Particle Export** for an explicit cache or selected settled-frame output. Since v2.009 the operator formerly named Export is Particle Export.

Optional gravity/load branch with a documented root filter: Cloth Bind's Send Out subsection can choose deactivated particles as its test condition. Its common Test Action can **Increment custom float** in a reserved channel `rootmask`, with **Send Out disabled**. New custom float channels default to 0, so free particles stay at 0 and deactivated roots become positive. Add Force below Slow and set its filter to `rootmask < 0.5`. This does not depend on undocumented material-ID propagation or a guessed activation-property enum. For a physical-gravity comparison use the documented real-world-value menu on the gravity spinner, then record its resulting native value at the chosen units/fps; do not assume the native number is -981 merely because the scene is in centimetres. Keep wind, noise, variation, spin and surface-area scaling off for this comparison. This optional branch adds two artist tasks: configure the test-action mask, then configure the filtered Force. It is not needed for the stone-driven baseline. [S31] [S43] [S45] [S46] [S63]

Cloth Bind creates constraints but the global Particle Bind Solver and CCCS execute at the end of each time step; the visible operator order is not itself the entire solver schedule. [S46] [S51] [S52] [S53] [S58] [S72]

Important settings and channels

- Cloth activation is the primary root constraint. No custom root channel is required for the no-Force conditioning baseline. The optional Force branch explicitly creates `rootmask` using Cloth Bind's deactivated-particle test condition.
- Preserve/inherit mesh UVWs. Saving vertex indices to a named custom float channel is available for diagnostics or later correspondence; it is not necessary merely to simulate a blade.
- Use **Mass per-cloth = 1.0** for the proposed starting run if changing subdivision should keep total mass comparable. Mass values are asset calibration, not universal grass parameters.
- High stretch/shear resistance; a starting stiffness of 1 is a proposed inextensible-looking proxy, with lower bend resistance tuned to the fixture. Volume stiffness is unnecessary for an open ribbon. Disable tearing. Constrained stiffness is less dependent on step count, but the docs only say *somewhat* independent, and it does not change stiffness-1 bindings.
- Enable **CUDA collision solver**, **Object collisions**, and generated cloth mesh. Choose small positive cloth thickness and record it; do not assume the documentation's textual thickness label

supplies an exact shell/radius convention. Enable self/inter-cloth collisions only when needed, then add their cost. Self-thickness must stay below the relevant separation of adjacent triangles to avoid self-repulsion.

- Start the main simulation timestep at **1/4 frame** (or 1/8 for tighter/faster contacts) and bind-solver steps at a proposed 20-50 for the small strip; refine until changing them does not materially change the accepted pose. The docs warn dense cloth can require hundreds of steps, so the small-fixture starting range is not a guarantee.
- CCCS repulsion iterations of 3-5 are documented as often useful; dense layers may need 20-50. Do not blindly raise steps when geometry is impossible or starts intersecting.

Expected result and acceptance

The root patch should remain coincident with its original world-space coordinates; the first free region flexes with bend resistance, and the visible strip itself collides with the stone. Inspect both sides and thin edges, not only the camera-facing silhouette. Accept only after visible motion is negligible over a chosen interval, root error is below the declared tolerance, and no surface intersections remain at the chosen clearance. Those are proposed validation criteria, not a claimed built-in tyFlow convergence test.

Artist work count

This baseline has **16 numbered artist tasks**, including output. These are reproducible task counts, not measured mouse clicks or setup time:

1. Set centimetres, Z-up, 30 fps and frames 0-90.
2. Prepare and retain the segmented 30 cm by 0.3 cm source ribbon.
3. Assign the noncollinear base patch its root material ID.
4. Prepare the closed stone mesh and record its final transform.
5. Author the clear-to-final stone animation and fixed hold.
6. Create an empty flow and configure Birth Objects.
7. Configure Cloth Bind, activation, stiffness, UVs and Generate Cloth Mesh.
8. Add/configure Slow.
9. Add/configure Collision and its source list/envelope.
10. Set main timestep, bind Steps and CCCS quality.
11. Inspect frame 0 topology, root activation and initial clearance.
12. Advance through loading and the fixed settling interval.
13. Inspect and select an accepted frame, or record failure.
14. Write the required cache.
15. Load/hold the cache or produce the selected-frame static mesh.
16. Render/export/reopen the chosen output and check geometry/material data.

Calibration repetitions are additional and unbounded. The optional gravity/rootmask branch adds exactly two defined tasks, for **18**. A saved template removes repeated operator construction, not root tagging or initial-intersection checks. None of these tasks was executed in Max here.

Optional detailed blade: tyParticleSkin

Status: documented binding feature, proposed per-blade integration. Use this when a high-detail render asset should follow a low-detail simulated ribbon. A ribbon driver carries a surface frame more explicitly than a sparse centerline. It is separate from the physical solve and can still produce render-mesh intersections between driver samples.

1. Prepare one low-resolution ribbon proxy and one corresponding high-resolution blade in the same reference pose. Keep the root and overall surface close enough for sensible local weights. Use one proxy/output object per blade for the minimal recipe; this deliberately avoids unverified automatic propagation of blade IDs through a whole clump.
2. Simulate the low-resolution ribbon with A/B. At the **reference frame**, add **tyParticleSkin** to the high-resolution blade and add the proxy/flow input. If using flow as a mesh driver, use **Convert input to mesh** so vertex transforms derive from the ribbon surface rather than assuming centerline particle rotations were solved.
3. Use vertex weighting for a simple starting point; choose the documented Distance, Falloff and Max count controls to keep weights local. Keep **Weight all vertices** off while diagnosing unbound areas/cross-binding. Use its vertex selection mode if the high-detail root patch should remain outside the deformation or be handled explicitly.
4. Bind/reinitialize at the selected **Ref frame**, then preview the settled frame. If rest topology, reference pose or correspondence changes, return to that frame and **Reinitialize**; moving an external obstacle alone changes the simulation and demands a reset/re-solve, but does not by itself require new rest weights if the original rest mapping is unchanged.
5. Inspect tips, root transitions, folds, unweighted vertices and cross-blade influences. A low-detail collision-free proxy is not proof the detailed mesh clears every stone. Export the actual resulting render mesh or use Particle Export's **Include additional geometry** option where appropriate; the particle cache alone is not proof that an external modifier result was saved.

The modifier documents both **mesh Element ID** and **UVW Element ID** matching. Birth Objects documents saving element indices, and its own page points to those IDs for tyParticleSkin matching. However, this research did not verify a complete automatic custom-element-ID propagation through Cloth Bind's newly generated per-vertex particles. Therefore the minimal reproducible recipe is isolated per blade. A production batch route must explicitly generate matching IDs on both sides, inspect them, and use the documented ID filter; do not invent a channel name and imply it exists. **Weight all vertices may bind unmatched/unweighted vertices despite element matching**, a documented exception. [S50] [S51] [S56]

The modifier page's claim that PhysX tearing mode only works with tyFlow is contradicted by v1.004 release notes adding tyCache input with exported PhysX data. Do not repeat that stale blanket limitation as current fact. Neither this ribbon recipe nor this skinning recipe needs PhysX tearing. [S31]

Additional manual operations: 6 defined task groups per isolated high-detail blade: align proxy/render reference; add modifier/input; set ref frame/mode; set local weights/selection; initialize and inspect; configure render-mesh export. Bulk automation is a separate pipeline task, not a free consequence of dropping on the modifier.

Static fitting boundary

No documented one-button static equilibrium fitter was established for these rooted collision assemblies. `tyPush` can statically move selected spline knots outward from chosen meshes; its Simple spline mode, selected-knot restriction, distance/falloff and outward-from-mesh direction are documented. Excluding first two knots preserves those coordinates. That is a useful manual correction tool, but the documentation does not promise preserved length, a root frame, joint multi-obstacle convergence, collision-free connecting segments or a guaranteed rendered mesh clearance. Do not promote it to a two-stone fitting solver. `tyBend` similarly provides a manual deformation, not the complete contact solution. [S60] [S61]

SECTION 8

tyFlow: grass between two stones

These variants reuse the documented operators and starting controls in Section 7. Geometry dimensions and analytic clearance checks are proposed fixture data; neither assembly has been executed in Max.

C2: Both stones fixed from initialization

Use the JSON's three two-stone guides rooted at X=-2,0,+2 cm, with both final spheres present and fixed from frame 0. Each has 17 samples and 30 cm length. The middle guide is straight. Each side guide has a vertical first segment, three unit directions toward the gap and out of the common plane, then twelve vertical segments. The side directions are `(+0.35,-0.15,sqrt(0.855))` for the left guide and `(-0.35,+0.15,sqrt(0.855))` for the right. This gives separated paths through the gap. The arithmetic-only minimum ideal-sphere clearances are **0.3715224 cm, 0.35 cm and 0.3715224 cm** for left/centre/right, exceeding the proposed combined 0.25 cm guide/collider envelope. There is no DCC test claim. The same Flat angular target attempts to straighten the side guides into their obstructed rest directions. If the chosen constraints/envelope do not admit the intended route, report failure; a local gap condition does not exclude a different three-dimensional escape path.

The three-guide/two-stone route adds **3 tasks**, for **23**: create the additional side-guide/source associations; prepare the second sphere; add it to Collision and inspect all three paths/contact envelopes. A template reduces repeated operator construction, but asset preparation and safe initialization remain real work. Counts are defined workflow tasks, not observed clicks, timing or a claim these operations were executed.

B: Direct ribbons with explicit conditioning

Status: same documented components, proposed two-obstacle assembly. Use **three 30 cm by 0.3 cm ribbons** rooted at X = -2, 0 and +2 cm. The two closed spheres have **radius 6.4 cm** and final centres **(-6.75,0,12)** and **(+6.75,0,12) cm**, giving a 0.7 cm centre gap. Start the stones at X = -30 and +30 cm at frame 0; move them toward their respective final transforms by frame 30, then hold to frame 90. Add both to the **same Collision list** before solving. This again has a conditioning phase; only the subsequent settling pass has both stones fixed. Keep root patches below and outside the spheres.

The second stone adds a competing contact constraint. Collision distance, strip width, root frame, length and bending resistance must admit a feasible configuration. A narrow gap smaller than the effective strip/collision envelope can be impossible. A ribbon can turn edge-on because its full strip geometry and frame are represented; that is a reason to prefer this route over a conservative cylindrical guide envelope. Do not silently reduce collision radius, unpin roots or stretch the strip to manufacture apparent success.

Solve both stones together. Add inter-blade collision only after the single-blade case works; diagnose impossible starts or envelopes before increasing iterations.

Preparation difference: add four tasks to A, for **20**: identify the three root-tagged ribbons; prepare the second sphere; animate its clear-to-final path; add it to Collision and inspect all associations/envelopes. Exchanging the approach order is an additional history-dependence test. After an edit, revise any invalid initialization and reset; a local instantaneous re-fit is not established.

SECTION 9

tyFlow: rigid pebbles

Use Max system units and display units of **centimetres**, Z-up, 30 fps, frame 0 as initial state, evaluate through frame 90 initially. Shared physical sizes: pebbles 20-40 mm; leaf 80 × 40 mm; curb 150 mm high. Do not label native Force/PhysX gravity controls as calibrated SI units without a local unit test. For a cross-application comparison, use the documented **real-world value menu on the gravity spinner**, then record the resulting native value for these units and frame rate. Official history documents that menu and subsequent unit-scaling/strength fixes; the current Force/PhysX parameter pages do not supply an SI conversion formula. Do not type -981 into a native control merely because scene units are centimetres. If the recipe is instead run with **-1.0**, label that as an optional native-value visual comparison, not matched Earth gravity. [S31] [S67] [S63]

Count each numbered item below as **one artist task**, not one mouse click. A task can contain multiple operator parameter entries. Modelling/import/cleanup of an arbitrary production asset is variable and additional; the listed fixture-preparation tasks explicitly include it rather than treating it as automatic. Counts are planned task counts, not observed timing or measured click counts. No claim that any software completes this in a fixed number of clicks.

For comparisons with a Houdini recipe whose first frame is 1, interpret Max frame 0 and Houdini frame 1 as the same initial time t = 0; compare elapsed time rather than matching the displayed frame number. Keep a saved source scene before each bake. Record version, scene units, frame rate, source object names, start/end frames, all settings changed from factory defaults, and final accepted frame. Initialize with disjoint bodies, valid normals and no root/leaf inside a collider. These three workflows have no roots: no root-position anchor and no root-orientation clamp. A rigid leaf is internally rigid as one body but **free to translate and rotate**; this is distinct from a rooted blade's anchored base.

TF-P: Rigid pebbles

Documented method: particle shape meshes converted to PhysX rigid bodies; static scene meshes supplied through PhysX Collision. **Proposed complete event:**

```
Birth Objects → PhysX Shape → PhysX Collision → Mesh → Display
```

No PhysX Bind, Cloth Bind, Particle Bind, guide, skin, constraint attribute or Force operator is required. Gravity is supplied **only once**, by the tyFlow object's PhysX World controls. Mesh/Display provide output; operator order is authoring order, not a claim about undocumented internal solver scheduling.

Artist tasks (12 setup/solve tasks + 5 common output tasks = 17)

1. Create/import a 1 m terrain patch with visible irregularity; weld/repair holes where solidity is intended, remove degenerate faces, orient its visible top consistently. Preserve one fixed, time-independent mesh for collision. For an easily inspectable fixture use a 40 × 40 grid with broad hills and depressions up to 50 mm; this topology recommendation is proposed geometry preparation.
2. Create one closed low-resolution pebble prototype of diameter 30 mm (within 20-40 mm range). Keep its pivot centred, use positive uniform scale, and use the same geometry for collision and display in this first test. Convex approximation must match the visibly relevant silhouette.

3. Instance the prototype into a separated 4 × 6 arrangement above the terrain, with centres at least 40 mm apart and each lowest surface at least 10 mm above the highest local terrain point. Use several initial rotations. This is actual placement preparation; it is not hidden automatic packing.
4. Create a fresh empty tyFlow with only the operators listed in this recipe (remove template/preset operators). Set History-dependent, Time step = 1/4 Frame, Time scale = 1, reset mode Manual while building. Preserve factory defaults except explicit choices in this recipe. [S54] [S55]
5. Add Birth Objects, pick those 24 nodes, Inherit object geometry on, start 0, End enabled off, Split object elements off, Preserve normals on. Turn continuous Inherit position/rotation/scale/velocity off. Hide source instances for final rendering, preserving the prototype separately. Birth Objects already supplies meshes, so no redundant Shape operator is required. [S51]
6. Add PhysX Shape, Hull Type = Convex Hull, Display Hull on, Size Mult = 1. Disable automatic Box/Sphere overrides for this test. Choose a sufficient Max verts for the prototype without assuming render triangles become collision triangles. [S65]
7. In PhysX Shape set Restitution = .05, Static friction = .6, Dynamic friction = .5, Angular damping = .1, Linear damping = .05. Equal-sized test pebbles may use Mass override = 1 with Relative to volume off; later mixed-size tests should deliberately choose volume-relative masses. Keep position/rotation locks off. Start Penetrations = Process all; the input is intentionally nonpenetrating. These values are proposed starting parameters. [S65]
8. Add PhysX Collision, pick terrain, Hull Type = Mesh, Display Hull on; set restitution/friction equal to the pebble values. Mesh is appropriate here because the obstacle is fixed. A single convex hull for the terrain would bridge depressions and lose concavity. [S66]
9. Configure tyFlow PhysX: PGS, Default gravity on, gravity set using its real-world value menu (record the resulting native spinner value), default Ground collider off, Substeps = 12, Pos iterations = 8, Vel iterations = 2, CCD on. Retain factory sleep settings initially and record them. Enhanced determinism can be enabled for repeatability investigation, but exported caches are the distribution contract. The values 12/8/2 are chosen starting values; the docs specifically suggest 12+ substeps for high-fidelity cases and explain the controls. [S67]
10. Add Mesh = Triangle Mesh for the compatibility preview and Display = Geometry. Assign the pebble material to the tyFlow object. A later qualified Render Instances mode can replace Triangle Mesh for repeated rigid geometry. [S69] [S57]
11. Clear Cache at frame 0, then evaluate sequentially to 90. Inspect low-angle silhouette/contact, hull-vs-visible-shape agreement, absence of large initial impulses, and cessation of sliding. Extend the interval if any body is still moving; 90 is a test window, not a settling guarantee. [S70]
12. If bodies tunnel, increase temporal resolution/CCD; if jitter persists inspect contact offsets, initial penetrations and mass disparity before increasing solver iterations. Accept a frame only after geometric inspection; sleep alone is not a proof of correct contact. Then complete output tasks O1-O5 below. [S65] [S67]

Required data and geometry sharing

Birth Objects creates particle identity, transform and shape data. There is no Houdini-style attribute list the artist must author for this baseline. No custom float is required for a single prototype. Optional birth/source index tracking becomes important with a mixed source library or reference-object export; it must be mapped deliberately rather than relying on random exporter source selection. [S51] [S56]

Rigid integration changes transforms, so unchanged source geometry remains eligible for sharing. That eligibility is distinct from an output path preserving it: Mesh Triangle Mesh and tyMesher Combined mesh explicitly build combined geometry. Mesh Render Instances and tyCache Instances can share duplicates in compatible renderers; tyCache GPU instancing can share duplicates in Nitrous. [S69] [S62] [S71]

Lifecycle and terrain edit

This is time integration. History-independent mode restarts every frame; it is not a static settlement solver. Jumping to a future frame may calculate preceding states, so hiding timeline playback does not remove the simulation. Reset after terrain edits and solve again from saved start geometry. In Auto mode dependent edits may clear caches and resimulate to the current time; Manual retains cache and may freeze on critical changes, with explicit inconsistency warnings until reset. A frozen output cannot automatically re-settle. [S54] [S55]

Shape and failure distinctions

- Dynamic **Mesh** hulls become kinematic: a pebble configured that way does not fall as the artist expects. Use Convex Hull, or a deliberate Compound hull if concavity matters. [S65]
- A sphere is cheap but rounds the silhouette and contact behaviour. A box is similarly coarse. Show hulls before tuning friction.
- Compound hull voxel modes can discard excessive voxels when Max voxels is exceeded. This can degrade a collision representation even though the source mesh looks intact. Do not automatically choose an extremely detailed compound. [S65]
- SDF is optional current functionality, not automatically superior. Accurate SDF contact requires both participants to use SDF; resolution and minimum cell size affect memory and detail. [S65] [S66]
- Excessive separation may come from hull/contact inflation; exploding overlaps may be eased by velocity limits, but clean initialization is preferred for this recipe. Sticky penetrations are a separate approximation and can sink; do not silently use them as clearance correction. [S65]
- A change to the collision terrain requires recomputation; editing an exported mesh or moving its cached playback object does not rerun the original settle operation.

SECTION 10

tyFlow: fallen leaves at a curb

TF-LR: Rigid leaves

Documented method: same rigid PhysX pipeline. A thin leaf remains one rigid particle; it may rotate, slide, stack and lean, but cannot fold. **Proposed complete event:**

```
Birth Objects → PhysX Shape → PhysX Collision → Mesh → Display
```

No root anchor/orientation stiffness exists in this unrooted example. Keep all rigidbody position/rotation locks off. Do not add PhysX Bind: it would constrain bodies to each other and changes the test's meaning.

Artist tasks (14 setup/solve tasks + O1-O5 = 19)

1. Create the 80 × 40 mm visible leaf with UVs/material and an actual geometric outline. An alpha texture affects shading, not the collision silhouette; a rectangular card remains rectangular collision geometry.
2. Centre the leaf pivot and make a very thin **closed** collision/display leaf for the baseline, for example 0.5 mm thickness. Use the flat common fixture first; a mild pre-existing curl is a later comparison. This is a proposed robust proxy choice; no undocumented claim that every zero-thickness convex hull fails. Collision follows the closed geometry, so over-thickening changes the apparent resting height.
3. Create an irregular ground patch and a 150 mm-high curb with face at x = 0, paved side on x < 0. Build appropriate collision geometry (box curb and fixed mesh terrain are sufficient for rigid; a single clean ground/curb surface also works). Inspect the corner the leaf is expected to contact.
4. Place one leaf leaning toward the curb without touching it initially. In cm, local leaf length along X, rotate Y = -65 degrees, centre (-1.99, 0, 4.125). Endpoints are approximately (-3.68, 0, .5) and (-.30, 0, 7.75), safely below the 15 cm curb top. This provides a gravity-driven lean/contact test without adding a wind/aerodynamic model. No outcome was executed.
5. Duplicate three more leaves along Y with separation greater than 40 mm; vary rotations only after the single-leaf case works. Keep initial meshes mutually disjoint.
6. Create fresh empty tyFlow with only this recipe’s operators, History-dependent, time step 1/4 Frame, Manual reset while building, time scale 1.
7. Add Birth Objects with start 0, no frame-range birth, mesh inheritance on, transform/velocity inheritance off, Preserve normals on, split elements off. Each whole leaf is one rigidbody. [S51]
8. Add PhysX Shape Convex Hull; show it. Check whether leaf lobes/curl are meaningfully lost. If the convex fill changes the curb contact materially, prepare a Compound version as a separate comparison, not an invisible switch. [S65]
9. Choose low restitution (.02), friction (.6 static/.5 dynamic), modest damping (.1 angular/.05 linear), mass 1 for equal leaves, all locks off. Use contact tolerance smaller than visible leaf width and inspect the resulting stand-off. Retain factory contact/penetration values as baseline, record them, reduce only when visible inflation is demonstrated. [S65]
10. Add PhysX Collision with ground and curb; use Mesh for irregular ground and Box or Mesh for the curb. If hull types differ, use two PhysX Collision operators; that is **one additional operator task**, raising this recipe to 20 tasks. Collision shapes should be explicitly visible during setup. [S66]

11. Use the PhysX controls from TF-P: PGS, single global gravity set with its real-world value menu, built-in ground off, 12 substeps, 8 position/2 velocity iterations, CCD on. No Force operator. [S67]
12. Add Mesh Triangle Mesh, Display Geometry and material. Rigid leaf variants remain eligible for instance rendering if their geometry is unchanged. [S69]
13. Reset at 0, evaluate to 90, inspect both sides of each leaf and the vertical curb corner. Confirm that apparent leaf-curb contact corresponds to the collision hull rather than a large empty convex region. Extend sim time if needed.
14. Freeze only an accepted frame. If the desired look includes wrapping or folding across the curb, switch to TF-LD; parameter tuning cannot make a one-body rigid leaf deform.

Changes, freeze, export and failures

Move the curb, clear/reset the source simulation, and replay from its initial leaf placements. Auto/Manual behaviour is the same as TF-P. Use O1-O5 for cache and output. A single shared leaf can be exported as reference-object **instances** using the documented Object export mode; if using multiple prototypes, record a source index and set List index from custom float to avoid random mismatch. Exported static transforms do not carry a live obstacle dependency. [S56] [S55]

Common failures: convex shape bridges a curled underside or leaf lobes; scale/contact tolerance makes a thin leaf hover; very fast edges tunnel; coplanar or highly thin collision geometry is numerically difficult; zero-mass/kinematic mode prevents settling; sleep freezes a visibly incorrect pose. Remedies are respectively more appropriate compound/SDF comparison, explicit thickness/tolerance inspection, CCD/substeps, better prepared proxy/scene scale, correct dynamic shape, and visual acceptance before bake. The proxy/thickness/scale recommendations are our preparation choices; the shape, CCD, offset and sleep controls are documented.

TF-LD: Deforming leaves

Documented method: Cloth Bind converts shape meshes into a mesh whose vertex particles are bound together; the Particle Bind Solver solves those bindings, and CCCS supplies triangle-level cloth/object collisions. **Proposed complete event:**

```
Birth Objects → Cloth Bind (Shape to cloth) → Force → Collision → Display
```

Cloth Bind **Generate Cloth Mesh** supplies the visible cloth surface. It is not necessary to assign little render spheres to the rig particles. A Mesh operator is useful only for any remaining particle shapes intended to render; it does not replace Generate Cloth Mesh. Use no PhysX Shape or PhysX Collision in this leaf-cloth event. This is the cloth/particle-bind solver path, not PhysX rigid bodies.

Artist tasks (16 setup/solve tasks + O1-O5 = 21)

1. Prepare one 80 × 40 mm leaf as a segmented surface with clean triangulation, UVs and material. A proposed fixture is 8 segments along length and 4 across width, diagonals consistently triangulated. Production silhouette/veins/curl need deliberate geometry or shading; neither Cloth Bind nor an alpha map reconstructs them.
2. Preserve a duplicate rest/source mesh for rebuilding. Keep final visual thickness separate initially; a single-surface cloth is enough to test bending and corner contact. Record face/vertex topology before binding.

3. Build the 150 mm curb plus floor as one clean, welded L-shaped collision surface, or clean nonintersecting pieces. Avoid overlapping collider meshes at the corner. CCCS documentation warns that intersecting obstacles can force unsolvable cloth configurations. [S34]
4. Place the same leaning leaf fixture as TF-LR, with no initial mesh/collider intersection. Keep a positive gap exceeding the selected total collision thickness. This is important preparation, not a solver cleanup toggle.
5. Copy the leaf into a small mutually separated row. Start with one; expand to four after contact quality is understood. All vertices are mobile: there is no root group or pinned tip in this fallen-leaf example.
6. Create fresh empty tyFlow with only this recipe's operators, History-dependent, timestep 1/4 Frame, time scale 1, Manual reset while building. Use PRO and compatible NVIDIA/CUDA 12.8 hardware for CCCS; install the matching current CUDA modules for performance (not a hard DLL prerequisite since v1.132). [S64] [S33] [S54] [S31]
7. Add Birth Objects, pick the leaves, start 0 only, Inherit geometry on; transform/velocity inheritance off; split elements off. There is one source shape per leaf before cloth conversion. [S51]
8. Add Cloth Bind, Operation = Shape to cloth, Activate all; Mass per-cloth = 1.0. Enable Inherit mesh UVWs. Save vertex index to a named custom float channel such as `leaf_vertex` only if a later skin/export association needs it; direct cloth rendering does not require hand-authored index attributes. [S46]
9. Configure stretch/shear stiffness 1, bend stiffness .2, volume stiffness 0, Constrained stiffness mode, damping .05, length bias 1; keep tearing off. These are proposed comparison settings, not calibrated botanical values. Use a later stiffness map for a midrib if artistically needed; that is additional preparation. [S46]
10. Enable CUDA collision solver, Object collisions, Inter-cloth collisions, Self-collisions. Choose base thickness .05 cm and self-thickness .05 cm as test values; keep self-thickness below the relevant spacing between neighbouring mesh triangles. Excessive thickness can make the leaf repel itself. [S46]
11. In Cloth Bind set Generate Cloth Mesh on, At rendertime only off, shell postprocess off for initial contact validation; assign leaf material to tyFlow. This direct deformation renders unique leaf surfaces. [S46]
12. Add Force, built-in gravity set with its real-world value menu (record the native value), wind/noise 0, Relative to perpendicular surface area off. Keep force-velocity influence at 100%, variation 0, and spin influence 0 for this gravity-only contact test. No global PhysX gravity is used by this path, and no second gravity force is added. This is settling under gravity; no claim of physically calibrated leaf flutter or aerodynamic lift is made. [S63] [S52] [S31]
13. Add Collision, choose clean floor/curb mesh, Absolute radius = .05 cm, friction 50%, bounce 0, pass-through 0, collide with both faces, default virtual ground off, Evaluation timing = After secondary solvers. CCCS uses this operator's absolute radius for obstacle thickness; Shape/Scale radius is not the documented CCCS thickness source. [S72] [S46]
14. Set Particle Bind Solver Steps = 40 as a starting point; use deterministic partitioning for cloth, keep stepped force integration on. Set CCCS Repulse steps = 5; start with factory Impulse and IZ budgets, recording them. Mesh stiffness is still dependent on resolution and solve work; Constrained mode only reduces that dependence. [S52] [S53] [S59]
15. Clear Cache at 0, evaluate through 90, watch both surfaces, MAXScript Listener collision warnings and excessive stretching. Lower time step to 1/8 before escalating collision budgets

indefinitely. Do not directly reposition cloth rig particles after initialization to force a desired result; that violates the documented CCCS motion assumptions. [S34]

16. Accept a frame when contact and shape are visually appropriate; then optionally add a small visual shell using Cloth Bind's shell postprocess, recheck clearance, and complete O1-O5. The visual shell is not a guarantee that the original collision clearance accommodates it. Turning the result into a cached/static mesh changes its lifecycle, not the fact that it was obtained through a timeline simulation.

Binding/geometry data

The operator creates the cloth rig, bindings and a generated cloth surface. Stretch/shear/bend are separate settings. Per-cloth mass avoids changing total leaf mass merely by increasing vertex count. No root anchor is used here; Activate/deactivate settings would freeze vertex positions, but a single frozen vertex would still not describe a whole base orientation. Direct cloth needs no guide-to-mesh transfer. [S46]

If a lower-resolution driver is later used with tyParticleSkin, the driver-to-visible-mesh binding becomes a separate preparation and validation step. The visible skinned leaf can contact the curb differently from its driver; collision of the driver is not automatically proof of high-resolution mesh clearance. Section 7 supplies the tyParticleSkin setup and element-matching limits.

Known failures and documented remedies

- **Initial intersections / collision warnings:** start from separate meshes and clean colliders; raise insufficient collision-step limits only after removing impossible geometry. CCCS warns via MAXScript Listener when its limits prevent completion. [S34] [S53]
- **Jitter:** lower timestep; inspect oversized thickness; increase repulsion work where necessary. More global frames do not fix an overly large per-step move. [S34]
- **Stretchy leaf despite high stiffness:** increase Particle Bind steps; use Constrained mode; inspect topology/density. The official cloth FAQ notes that dense cloth may need hundreds of steps. 40 is merely the proposed fixture baseline. [S59]
- **Self-repulsion:** reduce self-thickness relative to mesh spacing; remove sliver/coincident geometry. [S46]
- **Pinned/direct-position movement causes unresolvable contact:** do not use a post-initialization position overwrite as an ersatz collision solve. CCCS assumes force/velocity-driven movement; the docs explicitly name position-changing operators as failure causes. [S34]
- **Dense layering never settles / slow impact zones:** inspect impossible confinement and budget warnings; reduce timestep, limit leaf count and improve clean separation. Shift+Escape can break heavy CCCS computation, after which refresh/reset is required. [S34] [S53]
- **Farm frames differ:** CCCS has no deterministic mode; cache locally and render the cache rather than resimulate on each worker. [S34] [S68]
- **Material visible but no geometry:** confirm Generate Cloth Mesh; use correct output flags and shader assignment. Shading opacity has no effect on simulated collision shape. [S46]

Geometry sharing and timeline

Each leaf has its own changing vertex positions; initially identical source leaves no longer generally share one identical deformed mesh. This is a geometric consequence, not an inference about tyFlow's private mesh allocation strategy. tyCache records cloth/deforming mesh data separately from rigid particle transforms. Use Include cloth geo on export. Combining data does not restore instancing; the exporter explicitly says Combine meshes per-frame disables particle instancing. [S62] [S56]

History-dependent evaluation is required for this recipe. History-independent mode is a per-frame reset, not a quasistatic equilibrium solve. Once an accepted frame is cached, a tyCache retimer can hold it without rerunning the solver. Editing the source curb requires reset/re-simulation and a replacement cache; the frozen cache has no live connection to the moved collider. [S54] [S55] [S62]

SECTION 11

Reset, cache, render and export

Houdini lifecycle and output

Task	Static Guide Collide	Vellum hair/cloth	RBD rigid bodies
Initialization	Rest curves, final SDF, root mask; no previous simulation state	Valid initial geometry, matching geometry/constraint points, initialized orientation for hair, clear colliders or disclosed conditioning	Nonoverlapping packed bodies, correct mass/shape/name, clear terrain/curb
Reset	Re-evaluate from original guides	Dynamic: reset and replay from start. Quasistatic: reset/recook current valid inputs through the internal frame budget.	Reset Simulation and replay sequentially
Change obstacle	Recook SDF and fit; validate	Rebuild valid initial/conditioning inputs. Dynamic replays; frame-local Quasistatic recooks the changed arrangement.	Reset/replay settlement; wake/reuse old state only as a deliberately different history
Freeze	Single-frame File Cache	Dynamic: Time Shift accepted frame plus File Cache. Quasistatic: cache the accepted current-frame cook directly.	Time Shift + packed cache or transforms with rest prototypes
Restart	Preserve original inputs and fitting controls	Vellum I/O geometry+constraints; save collider separately	RBD I/O supports geometry, constraints and proxies or efficient sim points + rest high-res
Render	Deformed mesh or native curves if that is the intended asset	Simulated geometry or explicitly decoded render mesh	Render geometry follows packed transforms/proxy solution
Share geometry	Fitted shapes differ; packing each does not restore common prototype	Distinct deformation usually requires distinct point data	Pack/instance source before copying retains shared geometry; names distinguish bodies
Export	Baked meshes, or an explicitly supported curve/deformation pipeline	Alembic/USD deformed meshes; UV/normal/velocity support depends on format/settings	USD native instances/PointInstancer or Alembic instancing where possible

File Cache's Load From Disk is intentionally a cache switch: editing upstream while it is on can continue showing old data. Disable it to recompute and use a new version. Time Shift evaluates another frame and is not a cache itself. Vellum I/O Geometry Only is adequate for rendering but omits constraints needed for resuming the same simulation. Its collider input passes through without being saved. RBD I/O can cache rest high-resolution geometry and only simulation points thereafter.

USD SOP Import explicitly offers native-instance and PointInstancer choices for packed primitives. Alembic has packed-transform and instancing options. Neither proves a target renderer supports arbitrary per-instance point deformation; verify the receiving application. For final static vegetation, an explicit cluster of reusable bent shape variants would preserve reuse within each

identical variant, but that is a proposed Cyrus implementation rather than a documented Houdini guarantee for unique solved meshes. Sources: [S23], [S24], [S25], [S26], [S27], [S28], [S29].

tyFlow rooted-grass lifecycle

Stage	Required practical action and evidence boundary
Initialize	Set birth and reference frame explicitly; use an intersection-free source pose; create root mask/material selection; verify bind topology and activation before simulating. CCCS FAQ specifically warns that starting intersections break collision assumptions.
Simulate	Advance a history-dependent timeline from the beginning. History-independent mode means a fresh simulation each frame; it is not a documented static equilibrium solver. Choose an accepted frame after settling/inspection.
Reset	Auto update clears cached simulation and recomputes after relevant changes. Manual update can preserve old cached frames and then produce inconsistent later frames; use Refresh/reset before judging the changed setup.
Move one stone	Mark prior result stale; restore the original source/reference state and reset. Recheck start intersections with **all** final stones. If the new stone overlaps the initial grass, revise the clear initial path or use explicit gradual conditioning; merely continuing from an old cached frame changes the experiment.
Freeze	A live frame-retimer hold is convenient but is not a persistent external cache. Export an accepted frame as mesh/objects for an immutable static asset. For an animated bake, export the required frame range.
Cache	PRO Particle Export can write tyCache; spline mode stores Spline Paths data and tySplineCache can reconstruct it. tySplines' own cached spline data is RAM data and is not saved to disk by itself. CCCS is nondeterministic, so bake a single accepted simulation before rendering across machines.
Render	Render the generated cloth mesh, tySplineMesher mesh or tyParticleSkin mesh. Generic Mesh/Trimesh is the documented Max mesh route. Hide source/proxy objects appropriately to prevent double rendering. Check that render-only subdivision/material modifications do not reset or change simulation.
Export	Particle Export offers objects, tyCache, mesh/point Alembic, VRayProxy and spline outputs. Include cloth geometry, spline paths or external additional geometry explicitly when needed. Set start=end to the accepted frame for a static result. Combined per-frame meshes lose instancing. An event-scoped export's additional geometry can include all qualifying flow geometry, a documented limitation.

Sources: main settings [S54], reset [S55], tySplines [S48], Particle Export [S56], rendering [S57], CCCS FAQ [S34].

Geometry sharing and instancing

The original source blade can remain one reusable source asset. Once individual blades bend differently, their render vertices are different. A guide or cloth cache can preserve a compact procedural/deformation representation, but it is not ordinary transform-only instancing of one unchanged mesh. The cited modifier/operator documentation does not establish automatic deduplication of independently deformed blade meshes. Do not assert a private implementation or exact memory multiplier.

Per-blade tyParticleSkin normally means separate deformation results and weight state even if source topology/material is shared. A combined exported mesh explicitly removes particle instancing. A Cyrus optimization could cluster similar deformations and create reusable deformed prototypes; that would be a proposed Cyrus feature, not an existing capability inferred from tyFlow's interface.

tyFlow output tasks O1-O5

These five tasks complete the TF-P, TF-LR and TF-LD numbered task counts. The grass recipes already include their own output tasks. They can be reused across a batch of setups once the artist understands them.

O1 - Choose accepted output frame and cache range. Record accepted frame F, source scene and the decision to publish static F or animation 0…F. Keep source/rest geometry available. This is an artist acceptance decision, not an automatic “settled” flag.

O2 - Configure and run Particle Export. Use tyCache, Start = End = F for a static result or 0…F for animation, World coordinates, output path, Create tyCache object on. Press Export; the operator does not write merely because the timeline runs. Preserve particle ID/transform/shape data and required UV/material channels. For deforming leaves enable Include cloth geo. Export particles can be off if only the generated cloth surface is wanted. Keep Combine meshes per-frame **off for rigid repeated shapes**; for large unique deformation streams choose per-frame combination only with full awareness that particle instancing is disabled. PRO is required for tyCache export. [S56] [S33]

O3 - Hold and isolate output. Load generated tyCache; enable the appropriate Particles and/or Deforming meshes elements. Enable retimer, By frame, Frame = F without animation, or clamp to F…F with Hold. Hide/disable the source flow for output only after confirming the cache is complete; retain it in the editable source scene. A tyMesher Combined mesh at explicit Frame = F with Particle Interface = Ignore interface can extract regular generated cloth geometry; Tools > Extract mesh makes a new scene object. This alternative loses the particle instance representation. [S62] [S71]

O4 - Verify rendering. Compare the held cache and source at F with the chosen renderer, material/UVW/normal data and bounds. Rigid repeated meshes can use tyCache Instances in a supported renderer; deforming mesh output uses actual geometry. Nitrous GPU instancing suppresses mesh modifiers and per-particle UVW overrides unless a fallback is used; render-instance mode also does not provide a modifiable combined mesh to stack modifiers. Use actual mesh output when a downstream modifier is necessary. Do not claim viewport instance success establishes every renderer's behaviour. [S69] [S57] [S62]

O5 - Export or hand off. Keep the tyCache sequence **and accompanying mesh/material/support files**. FREE/RENDER farm loaders can consume existing caches. For static rigid scene nodes, Particle Export Objects > Reference Objects > Instance with one prototype and Animate transforms off preserves source-node sharing; with multiple prototypes a matching custom-float source index must be provided. A Meshes export makes meshes instead. For deforming animation use tyCache or qualified Alembic mesh export, preserving UVs/material IDs and testing reimport. The exporter documents different importer compatibility; for a legacy Max Alembic importer it recommends Max's own exporter after Mesh+Edit Mesh, while current vendor importers may support its direct Alembic. This does not establish the exact Max 2027 importer's compatibility without a local test. [S56] [S68]

Motion blur caveat

For static final renders there is no motion to blur. If exporting animation, rigid instance transforms can carry transform motion blur. Animated particle mesh vertices do not automatically gain deformation motion blur through the instance path. The older Motion Blur FAQ's statement that tyCache lacks subframe support conflicts with v2.000 notes adding tyCache subframe data; use current release capability but qualify renderer interpolation locally. Fixed-count topology plus mesh output is the conservative deforming leaf route. [S73] [S31]

Renderer-specific qualification

For Corona, the current Chaos guide says to enable Particle Interface for tyFlow instancing; disable it to pass an ordinary mesh through Max modifiers. Test the chosen result path and renderer version. A renderer-specific rule does not prove a universal tyFlow export behavior. [S74]

Houdini SOP Import supports packed geometry conversion to USD native instances or a PointInstancer. The dedicated current Point Instancer LOP is marked Since 22.0; the older general import route and that newer authoring node must not be conflated. [S75] [S76] [S28]

Parameters that must be recorded locally

- **Gravity:** changed the main recipes from an arbitrary -1 native value to the officially documented gravity-spinner real-world-value action. Record the resulting number because the current textual Force/PhysX docs do not publish a precise SI conversion. The optional -1 variant is explicitly labelled non-SI. There is only one gravity source in each recipe.
- **Frame numbering:** Max birth/start 0 versus Houdini start 1 is a display convention; compare t = (frame - start) / fps. Reset to the configured birth/start frame before replay; replaying only from a final frame is not initialization.
- **Birth Objects:** animation inheritance toggles govern subsequent source motion. Documentation separately describes initial object-to-particle transform transfer, including the case where geometry inheritance is disabled. Keep initial source transforms and disable subsequent position/rotation/scale/velocity inheritance; inspect frame 0 before simulation. This is a documented interpretation, not an executed test. If the artist intentionally animates the source, moving particles out of the Birth Objects event is the separately documented way to end its animation updates.
- **Cloth surface output:** Generate Cloth Mesh explicitly creates the actual cloth surface; At rendertime only controls when. Mesh controls the rendering of particle shape meshes. Those are distinct documented representations. The recipe relies on the generated cloth geometry and its explicit export checkbox; it does not claim a verified renderer result without a local render.
- **Friction units:** PhysX Shape/Collision use coefficients (.6/.5 in the proposal); ordinary Collision is labelled Friction% (50 in the proposal). Cloth Bind's textual manual gives a Friction control and averaging semantics but does not explicitly state its UI unit/range; retain and record its v2.101 baseline instead of asserting an undocumented percentage conversion. These values should not be presented as physically identical across solvers.
- **Solver naming:** Main Settings > Time step; Particle Bind Solver > Steps; CCCS > Repulse steps/Impulse steps/IZ steps; PhysX > Substeps/Pos iterations/Vel iterations are distinct controls. A historical FAQ's generic max-collision-steps wording is not a new current UI field.
- **Baseline settings:** settings not assigned numbers in these recipes (such as CCCS Impulse/IZ limits, cloth friction and sleep thresholds) must be recorded from the chosen v2.101 scene.

User-saved custom defaults can differ. The full reproducibility manifest must include those values; these notes do not invent undocumented factory numbers.

- **Rigid sleep:** low velocity plus duration controls resting candidates, not contact correctness. No PhysX Bind is present, because the docs say bound rigidbodies ignore ordinary PhysX sleep thresholds.

Per-obstacle-edit task count

For either terrain or curb edits, the explicit Manual-mode update path has 6 tasks: (1) return the editable source scene to its configured start frame; (2) edit the obstacle and inspect its collision representation; (3) Clear Cache/reset; (4) replay and inspect to choose a new accepted frame; (5) export a new completed cache version; (6) point the output loader at that version, hold the accepted frame, and check the render. If source-object mesh data was edited and Birth Objects auto-update is disabled, its documented Update Cache is an additional task. Auto reset can automate the reset/replay trigger but still does not automate acceptance or replacing a published disk cache.

Houdini baseline record

The proposed trial now specifies RBD density (2000 kg/m³ pebbles, 500 kg/m³ thin rigid-leaf proxy) and uniform 1e-5 point mass for the deforming-leaf fixture. These are test values, not measured material data. Record H22 node versions, effective point/total mass, incoming friction/pscale, damping and sleep thresholds, plus any inherited settings. Use the same recorded scene when comparing revisions. Vellum point mass and density-derived mass are different modes; do not silently change total mass while refining topology. [S02] [S20]

SECTION 12

Artist work, failures and local acceptance

Numbered setup-to-output task counts

These are task groups in the written recipes, not measured clicks, setup time, or an efficiency benchmark. Some tasks combine several UI entries. Fixture/source preparation is included where numbered; production cleanup, material creation, calibration repeats and optional detailed bindings are additional. The Houdini recipes mainly begin from imported fixtures; the tyFlow lists explicitly include more fixture authoring. Do not rank the applications by these totals.

Houdini recipe	Tasks	Scope / main extra preparation
H1B static guide collision	9	One guide, one SDF, root mask and named mesh decode; no guaranteed hard frame.
H2B static two-stone candidate	12	Second SDF and merge; joint-contact validation.
H1C fixed-stone Vellum	16	Separate straight-rest constraints, clear initial curve and fresh orientations.
H2C two fixed stones	18	Three named guides and second merged collider; mesh-frame qualification remains required.
H1A conditioned hair	12	One clear-to-final collider history; fixed only during hold.
H2A conditioned hair	14	Two collider histories and joint solve.
H3 rigid pebbles	12	Packed copies, consistent names, static concave terrain; extra proxy work if needed.
H4R rigid leaves	12	Explicit thin-proxy preparation, naming and transformed render association.
H4D deforming leaves	12	Prepared surface, cloth constraints and contact validation; extra high-resolution skin excluded.

tyFlow recipe	Tasks	Scope / main extra preparation
C fixed-stone guide	20	Clear spline, explicit root mask, bind topology, spline output and mesh mapping.
C2 three guides / two fixed stones	23	Additional guide associations, stone and joint contact inspection.
A conditioned direct ribbon	16	Root material patch, collider animation, generated cloth output.
B three ribbons / two conditioned stones	20	Additional source association and second collider history.
Optional filtered gravity on ribbon	+2	Deactivated-particle test action and a filtered Force operator.

tyFlow recipe	Tasks	Scope / main extra preparation
Optional detailed tyParticleSkin blade	+6	Reference alignment, modifier/input, reference mode, weights, binding check, final-mesh export.
TF-P rigid pebbles	17	12 setup/solve tasks plus the five output tasks in Section 11.
TF-LR rigid leaves	19	14 setup/solve plus five output; 20 if separate collision operators use different hull types.
TF-LD deforming leaves	21	16 setup/solve plus five output; surface topology and collision cleanup are explicit.

What changes after moving an obstacle

The practical update is shorter than first setup, but it still has state. Static Guide Collide requires the final SDF to recook, followed by root/length/contact checks and a replacement static cache. Vellum requires a valid initial geometry or conditioning history, reset/recook or replay, acceptance and replacement output. For the tyFlow terrain/curb settling recipes, the six Manual-mode edit tasks are listed in Section 11. Rooted grass can additionally require a new clear initial guide or revised conditioning motion. Moving an obstacle does not itself require recapturing unchanged rest mesh weights; changing source topology or reference correspondence does. A loaded disk cache is deliberately stable until replaced.

Houdini failure diagnosis

Observable failure	Evidence-backed cause/remedy and practical limitation
Root stays in place but blade pivots freely	Position pin was set without orientation pin. Use the separate Vellum Orientation Pin Type. Root position, tangent and twist should be reported separately.
Guide appears clear, render blade cuts the stone	A centreline/capsule is only a proxy. Increase radius to bound blade width, improve decoding/frame, or simulate a surface. Validate render geometry. This geometric implication is an inference, not a claim that native hair collision checks an arbitrary mesh.
Grass drifts between neighboring guides	Spatial capture blended across blades. Use authored names with Path Deform, per-blade Wire Capture groups/loops, or a piece-restricted surface cage. Standalone Point Deform is explicitly unsuitable for a wire model.
Strip twists/flattens unexpectedly	Up/forward axis conflict, ambiguous frame, or collision `pscale` used as a transform scale. Set nonparallel axes and control/sanitize curve transform attributes. Use explicit normals for a 3D wire frame.
Stone teleport causes explosions or grass remains embedded	Invalid initial contact/history. Reset to a valid configuration; use disclosed conditioning or pre-bend. Do not treat initial overlap-reduction/disable mechanisms as a complete static depenetration solution.
Two-stone gap never resolves	Channel too small, root lies in obstacle, or obstacles intersect. Reposition/exclude the blade or change the request; more passes cannot create physical space. The SideFX intersecting-collider guide illustrates changing collider geometry to create space, not solving an impossible hard collision.
Narrow gap disappears in static fit	VDB voxel/band resolution loses details. Refine voxels and inspect signed-field topology; do not assume final polygon gap equals SDF gap.
Vellum collision silently fails	Solver visualizers show failed self/external points. Documented disabling can affect incident edges/triangles. Inspect flags and reset valid inputs; increase substeps/collision/constraint passes only for genuine resolution issues.
Highly elastic or unstable leaf/grass	Stiffness and convergence depend on discretization/time stepping. Add substeps or constraint iterations; adjust damping judiciously. H22 orientation quasistatic mode needs its own settings.
Leaves gain intersections only after smoothing	Post-process can create collisions. Minimize geometric changes or use documented Detangle with inspection; it is limited and large corrections can degrade shapes.
Pebbles float above valleys	Convex terrain hull bridges relief or oversized collision padding. Use static concave terrain and appropriate padding/shrink.
Rigid leaves behave as thick blocks	Collision hull/padding differs from render silhouette/thickness. Inspect the actual Bullet shape; prepare a valid thin proxy and scale tolerances.
Wrong render piece follows a body	Nonunique/inconsistent `name` types or proxy mismatch. Use consistent names on render/proxy streams.
Moving a stone seems to do nothing	Old File Cache Load From Disk or old simulation cache. Recompute/reset and version the cache; expose cache status to artists.

Sources: [S01], [S02], [S03], [S09], [S10], [S11], [S12], [S13], [S14], [S15], [S17], [S18], [S19], [S20], [S21].

tyFlow grass failure diagnosis

Visible failure	Likely cause / documented limitation	Remedy and limit
Root stretches or drifts in bind solve	Root was position-followed but still active; Particle Switch docs warn this causes stretching.	Deactivate intended anchor particles; inspect all root coordinates. Filter force/position operators that could move them independently.
Blade swivels around a fixed root	One positional pin does not constrain tangent.	Pin first two guide samples; for full frame use a fixed noncollinear ribbon patch.
Blade twists/flips although two guide points are fixed	Collinear points do not determine cross-section twist; core bind solver does not update rotations.	Use a ribbon driver/full base patch or an explicit tested frame construction. Do not label a Rotation operator as physical torsional stiffness.
Guide segments cut through the stone between particles	Only particle collision was configured, or insufficient sampling/collision path.	Enable Particle Bind's CUDA segment collision and Object collisions, with Collision in same event; inspect guide radius and sampling.
Render blade intersects while guide clears	Centerline and mesh envelopes differ; smoothing/mesh mapping introduces offset.	Increase conservative guide radius/resolution, inspect rendered mesh, or use direct ribbon collision.
Cloth explodes, stays tangled, CCCS stalls	Initial intersections; interpenetrating obstacles; impossible trapped configuration.	Start clear, adjust geometry or intentional conditioning; don't expect iterations to untangle an impossible state.
Root/helper/Find Target produces unstable contacts	Direct position changes can bypass CCCS assumptions.	Prefer compatible dynamics and fixed initial anchors; do not teleport colliders/points. FAQ names sticky Object Bind and direct position control as problematic cases.
Jitter or contact leakage	Steps too large, coarse discretization, weak/inadequate solve, scale/thickness mismatch.	Smaller timestep (FAQ suggests 1/4 or smaller), more appropriate bind/CCCS steps, meaningful thickness and topology. Re-test a single blade before a clump.
Cloth repels itself before contact	Self-thickness too large relative to neighboring triangles.	Lower self-thickness below the relevant local spacing; distinguish self thickness from overall collision envelope.
Slow/no convergence in the two-stone gap	Clearance smaller than effective thickness, overconstrained base/length, dense contact.	Show clearance failure; reduce physical envelope only if the artist intends it, or change root/path/stone geometry. CCCS iteration warnings appear in MAXScript Listener.
Strange render splines/cross-links	Proximity binding joins different blades; angular bind IDs included in Spline Paths.	Bind siblings only, filter the original segment binding ID, Weld/Sort bindings, retain stable ordering.
High-detail vertices follow neighboring blades	Weight search is spatial; ID/element correspondence missing; Weight all vertices overrides matching.	Isolate one blade, constrain radius/count, verify matching IDs, disable Weight all vertices during diagnosis.

Visible failure	Likely cause / documented limitation	Remedy and limit
Different farm frames show different settling	CCCS nondeterminism; CPU/GPU paths differ.	Bake the accepted result once and distribute cache; ordinary bind-solver determinism setting is not a CCCS determinism guarantee.
After moving a stone the old result stays visible	Manual update or stale cache.	Clear/refresh from initial frame, re-simulate, inspect, then replace published cache.
Cache/export misses deformed high-resolution geometry	Export only contains particle channels/proxy/splines.	Export actual final meshes or explicitly Include additional geometry; verify one reopened/exported frame.

Sources: [S34], [S36], [S37], [S38], [S46], [S47], [S48], [S49], [S50], [S51], [S52], [S53], [S54], [S55], [S56], [S59].

The recipes are small enough to validate locally before adapting a production vegetation library. Keep the same source and obstacle fixtures while changing one setting at a time. The purpose is to expose what the artist must prepare and what state the software carries between edits.

Minimal local run order

1. Run a single blade/leaf/pebble before the small repeated set. Display its guide or rigid hull beside the visible geometry.
2. Test the position anchor and orientation control independently. For a curve, intentionally permit base rotation once; then enable the orientation constraint and verify the difference.
3. Run the two-stone case with both obstacles participating simultaneously. Compare the result with separate sequential edits, without treating either as a unique physical answer.
4. Move an obstacle away. Recompute from the declared starting state and confirm that any previous deformation or settled transform is handled according to that method's documented lifecycle.
5. Hold a frame, close/reopen the scene with the cache, and render it. Then export/reimport the same frame and check the geometric and material data.

What would invalidate a recipe locally

An unknown node version, missing licensed solver path, absent CUDA capability, mismatched topology, missing per-piece correspondence or a collider that cannot be represented as selected is a setup failure. Initial penetration, unresolved contacts, unacceptable strain or a root/frame violation is a geometry/solve failure. A correct viewport pose that changes at render or export is an output failure. Preserve these categories in the test record rather than averaging them into a visual-quality score.

A failure can be informative: a straight sparse guide that leaves a twisted or intersecting ribbon identifies a decoder/frame problem. Increasing collision iterations will not fix an incorrect binding. A pebble floating over a valley represented by a convex hull identifies the collider shape. Increasing friction will not restore the missing valley.

Reusable preparation versus repeated work

Root selection, guide creation, frame convention and mesh binding should be reusable for an unchanged vegetation source. Rebuild them after topology or correspondence changes. Moving a stone normally requires collision/solve recomputation, not rebinding a mesh whose rest topology is unchanged. A cache loaded from disk is a deliberate publication; it does not silently inherit edits to the source obstacle.

Rigid settling has the smallest deformation-preparation burden: suitable closed shapes, separated starting transforms and an accurate terrain collider are enough for the proposed baseline. Deforming leaves require surface topology and thickness decisions. Rooted grass adds root/frame identification and, for the guide route, a second geometry representation plus a decoder. This is the practical difference that a Cyrus workflow should reduce without hiding it.

Local acceptance record

Record versions, units/fps, topology/IDs, collider shapes, root groups, constraint modes, thickness, solve budgets, initialization branch, accepted frame, cache path and renderer. The following checks are proposed, not executed measurements:

- Compare root coordinates and base frames; distinguish a single position pin from a full clamp.
- Check guide segments and decoded mesh edges against every obstacle; measure length/shape change.
- Move a stone into, out of and across contact; reset from the declared initial state and compare with a fresh setup.
- Freeze, reopen, render and export/reimport; verify UVs, materials, normals, bounds and instance representation.

SECTION 13

Evidence register and source links

Official documentation and example pages retrieved on 24 October 2026. Notes identify evidence scope and version boundaries. Exact scene assemblies are proposed recipes unless explicitly marked otherwise.

What was and was not executed

Executed here: public-document retrieval; downloaded example hashing; decompression and inspection of saved HDA/OTL network records; report generation; ideal-sphere/line-segment geometry arithmetic on the shared JSON fixture. No Houdini or 3ds Max executable was available. No DCC simulation, render, bake, export/reimport, timing or memory benchmark was executed.

All scene-level recipes remain P. An official feature description is D. An official downloadable network inspected as text is I. No exact four-case vendor suite was found and executed. Numeric convergence and visual success must be recorded by the artist who runs the scenes.

Inspected Houdini examples

All are public SideFX assets, downloaded directly from documentation URLs. `houdini_evidence/example_downloads.json` records full URLs, sizes, SHA-256 hashes. Each extracted archive has an `archive_manifest.json`. The HDA/OTL bytes were decompressed and node `.def`/`.parm`/notes read; that is a structural inspection, not proof of executed behavior in the current build.

Example	Inspected evidence	What it does not prove
`examples/nodes/sop/vellumsolver/HairOrient.hda`	Root first-vertex expression; hair constraints; Guide Deform transforms `P orient`; deformed target geometry to Solver input 1 while original constraints feed input 2; root soft positional pin and same orientation rule.	Its inspected solver has **collisions off**. It is not a grass/stone contact or hard-root example.
`examples/nodes/sop/pathdeform/PathDeformPiece.hda`	Named box/sphere/tube source pieces, named curves; `usepiece=on`, `pieceattrib=name`.	It demonstrates association, not collision handling or the proposed no-stretch blade settings.
`examples/nodes/sop/pathdeform/PathDeformBasic.hda`	Archive downloaded and unpacked.	No simulation or detailed behavior validation was performed.
`examples/nodes/sop/wirecapture/BasicWireCapture.hda`	Downloaded and inspected capture network structure.	Not a vegetation collision test.
`examples/nodes/sop/wiredeform/TwistWireDeform.hda`	Authored N and tangent; Wire Deform `xformframe=tangentnormal`, normal recomputation.	Does not verify an automatic conversion from Vellum quaternion frame to arbitrary leaf/blade UV axis.
`examples/nodes/sop/rbdbulletsolver/RBDBulletSolver.otl`	Archive downloaded and unpacked, node/parameter structure available.	Not labeled or executed as a pebble/leaf fixture.

HairOrient SHA-256: `20859249f91598cc28c61bf2abe119bfa3bf0c03ad198fc93efe74cc455b9d9a`.

PathDeformPiece SHA-256: `2f2b2083b9fa9e7df64fce23a2fd2666ac6cba8e13f7276b1922512d2942b448`.

TwistWireDeform SHA-256: `1ebf2e1d897e2f74e691c97191b4f4658e0faaff7b12562937f6a77369618655`.

Exact downloaded-asset identities

HairOrient.hda - 82484 bytes. SHA-256: `20859249f91598cc28c61bf2abe119bfa3bf0c03ad198fc93efe74cc455b9d9a`. [S80]

BasicWireCapture.hda - 20163 bytes. SHA-256: `10beecb49410e47de4930f7fc489c8a4b5695179512bcd91f3a3b1cc830ba78a`. [S81]

TwistWireDeform.hda - 21059 bytes. SHA-256: `1ebf2e1d897e2f74e691c97191b4f4658e0faaff7b12562937f6a77369618655`. [S82]

PathDeformBasic.hda - 45849 bytes. SHA-256: `6ea18a397662b3a5c92ac8da599e892c0907555da7b00098834be42e00bcec12`. [S83]

PathDeformPiece.hda - 38915 bytes. SHA-256: `2f2b2083b9fa9e7df64fce23a2fd2666ac6cba8e13f7276b1922512d2942b448`. [S84]

RBDBulletSolver.otl - 180744 bytes. SHA-256: `d605abc07d1220e59cf52b5d0d2a0a5f7ea9ee730c505aa092cfb277d0572f9d`. [S85]

tyFlow example scope

Official history names `tyFlow_cloth_CCCS_splineSoup_001.max`, `tyFlow_cloth_CCCS_papers_001.max`, and `tyFlow_cloth_CCCS_book_001.max` in Example Scenes 016. The current download page lists an example archive dated 2 September 2026. Names/listings were verified, but those .max binaries were not inspected or run. They are discovery leads for a local artist, not evidence that this report's grass/stone or curb/leaf scene already succeeded. [S31] [S64]

The companion `CyrusScatter_Common_Grass_Initialization.json` contains both coordinate systems and the analytic preflight checks. Its original research filename in the recipes is `common_grass_initialization.json`; the content is identical. It is neutral input data, not a .hip/.max scene or a solver result.

Primary source register

Each reference below identifies the supported public capability. Repeated URLs are consolidated; recipes combine these components with explicitly proposed geometry and parameter choices.

[S01] Guide Collide With VDB

SideFX | Since 16.0 | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/sop/guidecollidevdb.html

Static curve collision against signed-distance VDBs. Extra VDBs enter fourth input; Collide With Skin is optional. Curve mask, surface offset, push range and push amount are documented.

[S02] Vellum Constraints

SideFX | Since 17.0 | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/sop/vellumconstraints.html

Hair creates edge stretch and bend/twist constraints; Cloth creates edge and triangle bend constraints. Geometry/constraints travel as two matching streams. Pin Type Stopped preserves mass while fixing position; Orientation Pin Type None/Same as Position/Soft independently controls outgoing segment orientation.

[S03] Vellum Solver

SideFX | Since 17.0; current page H22 | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/sop/vellumsolver.html

Inputs 1 geometry, 2 matching constraints, 3 collision geometry. Dynamic simulation versus frame-local Quasistatic internal forward simulation. Full vs Minimal mode restrictions; reset/cache controls and collision diagnostics.

[S04] Vellum Hair shelf tool

SideFX | Vellum-era shelf workflow | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/shelf/vellumhair.html

Hair simulates polygonal curves with length/orientation/twist; root group uses first point. Unbranched curves are baseline; branches require special splitting/welding preparation.

[S05] Houdini 22 Vellum news

SideFX | Houdini 22.0 | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/news/22/vellum.html

New quasistatic bending/twisting orientation solve for hair. New Angular Damping, different stiffness/damping expectations for the new method.

[S06] Guide Deform

SideFX | Current node Since 22.0 | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/sop/guidedeform.html

Current interpolation methods for grooming. Preserve Shape rod relaxation and Lock Roots. No documented obstacle collision input for Preserve Shape; do not claim stone contact from it.

[S07] Deforming guides and hair

SideFX | Houdini 22.0 workflow | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/fur/guidedeform.html

Official Capybara Sim and Deform shelf workflow: reduced guides, utility, Guide Deform, Vellum, dense groom deformation. Dense groom inputs rest/animated skin and rest/simulated guides. Solaris render-time guide deformation is for the groom workflow, not arbitrary mesh skinning.

[S08] Guide Utility

SideFX | Since 22.0 | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/sop/guideutility.html

Roots group creation and snapping controls; distinguish current H22 tool from older pipelines.

[S09] VDB from Polygons

SideFX | Since 12.5 | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/sop/vdbfrompolygons.html

Signed distance field, voxel resolution, interior/exterior bands, Fill Interior. Unsigned distances do not supply inside/outside; small features can be lost below voxel resolution.

[S10] Path Deform

SideFX | Node Since 18.0; current controls may be later | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/sop/pathdeform.html

Input mesh and spine curves; piece-attribute association. Forward/up directions and curve framing; geometry length versus curve-length mapping. Orient Using Point Attributes includes pscale and scale, so collision radius must not inadvertently rescale render mesh.

[S11] Wire Capture

SideFX | Current official documentation | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/sop/wirecapture.html

Captures mesh to rest curves with primitive ranges/radii and subgroup restrictions. Official BasicWireCapture downloadable example.

[S12] Wire Deform

SideFX | Current official documentation | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/sop/wiredeform.html

Captured geometry/rest curves/deformed curves inputs; curve primitive counts/order must agree. Explicit tangent plus attribute normal local frame; normal recomputation. Fast recache requires caution if topology/attributes change.

[S13] Point Deform

SideFX | Since 14.0 | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/sop/pointdeform.html

Mesh/rest/deformed lattice inputs, IDs and piece restriction. Explicit warning that wire models do not provide stable transforms. A piece attribute avoids cross-piece capture but does not solve the wire-model limitation.

[S14] Vellum attributes

SideFX | Current H22 attribute reference | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/vellum/vellumattributes.html

pscale is point collision radius. stopped bits distinguish translation and rotation; orient and angular state. Initial-overlap and disabled-collision attributes can affect incident edges/triangles.

[S15] Vellum tips and troubleshooting

SideFX | Current H22 guide | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/vellum/vellumtips.html

Topology/ID consistency for continuous collision. Thickness versus edge length; substeps/constraint iterations/collision passes. Excessive post-collision passes can create protrusions; damping affects convergence.

[S16] Vellum Brush

SideFX | Since 18.5 | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/sop/vellumbrush.html

Interactive Damped/Settle simulation modes, input-frame freeze and live simulation. Reset All Changes and Set Rest State differ; changing rest state is not just caching.

[S17] Cloth simulation

SideFX | Current H22 guide | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/vellum/clothsimulation.html

Five substeps suggested for cloth, bending affects wrinkles, static friction controls sliding. New cloth configuration may reset thickness to 0.01. Duplicate IDs and inconsistent winding need cleanup.

[S18] Fixing intersecting collision geometry

SideFX | Current H22 guide | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/vellum/intersect.html

Intersecting animated collision meshes can leave cloth no valid space. Official example changes collider shape with soft/hard pinned Vellum collider geometry to make space. Not evidence a solver can satisfy mutually impossible hard obstacles.

[S19] Vellum Post-Process

SideFX | Since 17.0 | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/sop/vellumpostprocess.html

Weld/smooth/subdivide/extrude processing, failed-collision visualization. Smoothing/subdivision can introduce intersections; Detangle is limited.

[S20] RBD Configure

SideFX | Since 18.0 | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/sop/rbdconfigure.html

Packed simulation attributes and render/proxy handling. Convex/concave/primitive collision representations; collision margin and shrink. Active/animated/deforming/sleeping, density/friction/bounce.

[S21] RBD Bullet Solver

SideFX | Since 18.0 | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/sop/rbdbulletsolver.html

Inputs render geometry 1, constraints 2, proxy 3, external collision 4, guides 5. Consistent name classes and render/proxy correspondence are required. External Static/Animated/Deforming types; Collision Shape Convex Hull/Concave and others.

[S22] Simulating at the SOP level

SideFX | Current official SOP RBD guide | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/destruction/sop_sim.html

SOP-level RBD workflow; custom DOP editing can require Houdini FX license.

[S23] File Cache

SideFX | Current 2.0 Since 19.0 | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/sop/filecache.html

Static versus time-dependent cache; sequential simulation caching. Load From Disk and cache versioning; upstream edits require recomputation.

[S24] Vellum I/O

SideFX | Current 2.0 Since 19.5 | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/sop/vellumio.html

Geometry and constraints storage versus geometry-only output. Third collision input is passed through but NOT cached.

[S25] RBD I/O

SideFX | Current 2.0 Since 19.5 | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/sop/rbdio.html

Render/constraints/proxy storage and simulation-points option. Rest high-res cache plus transforms is an efficient output route.

[S26] Time Shift

SideFX | Current official documentation | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/sop/timeshift.html

Evaluates a different/fixed frame; explicitly does not cache by itself.

[S27] Copy to Points

SideFX | Current 2.0 Since 18.0 | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/sop/copytopoints.html

Pack and Instance packs prototype before copying so geometry can be shared. Piece attribute chooses source geometry; target attributes drive transforms.

[S28] SOP Import

SideFX | Current official Solaris documentation | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/lop/sopimport.html

Packed primitives may become native USD instances or a PointInstancer. Other unpack/xform modes differ; material data requires deliberate handling. Unique deformed geometry does not automatically become one shared prototype.

[S29] Alembic ROP

SideFX | Since 12.0 | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/out/alembic.html

Packed-transform handling and Use Instancing Where Possible. Static or time-range geometry export; named hierarchy/attributes and face sets. Option does not guarantee receiver support or reuse of distinct point deformations.

[S30] Guide Process

SideFX | Since 16.0 | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/sop/guideprocess.html

Masked geometric guide edits may author a pre-bend; not claimed to provide obstacle contact guarantee.

[S31] Version history

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/download/version/

Observed current v2.101 (2026-10-08); v2.004 Max 2027; v1.132 optional supplemental DLLs; v1.131/v2.000 CUDA 12.8; historical tyParticleSkin/tySplineMesher/CCCS and named vendor example releases. History v0.16025 documents a real-world-value menu on gravity spinners; later notes fix unit scaling and a 2x strength bug.

[S32] Installation

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/download/installation/

3ds Max 2020+ and host-version-matching installation binary.

[S33] FREE license

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/license/free/

FREE disables GPU acceleration, multithreading and tyCache export; other features allowed for personal/commercial work.

[S34] Simulation: CUDA Cloth Collision Solver FAQ

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/faq/CUDA_collisions/

No complete CPU fallback; nondeterminism and need to cache; initial intersections and direct position control can fail; timestep/thickness/impossible configurations and listener/abort diagnostics.

[S35] System requirements

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/download/system_requirements/

Distinguishes ordinary binding solver OpenCL/CPU fallback from NVIDIA-only CCCS and PhysX GPU/CPU capabilities.

[S36] Particle Bind

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_operators/particles/particle_bind/

Creates bindings solved globally; sibling proximity 1 adjacency; separate binding IDs; stiffness modes; CUDA segment collisions and segment radius; object collision requires Collision in same event and only uses absolute radius for collider thickness.

[S37] Angle Bind

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_operators/particles/angle_bind/

Runs after existing binds; ID filter/new ID; angular/area stiffness; Current or Flat 180-degree rest-angle target; warning on disabling Invertible.

[S38] Particle Switch

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_operators/particles/particle_switch/

Deactivated particles treated as infinite mass by bind solver; warning that not deactivating intended static anchors causes stretching.

[S39] Object Bind

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_operators/particles/object_bind/

Independent position/rotation/scale following, transform/surface lock and springs.

[S40] Rotation

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_operators/particles/rotation/

Core particle bind solver does not update rotations; sibling and neighbor rotation reconstruction modes exist.

[S41] Hair Bind

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_operators/particles/hair_bind/

Attaches particles to Ornatrix strands; Position Hair must first save location/strand transforms in custom TM channels.

[S42] Birth Spline

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_operators/particles/birth_spline/

Birth by knots/distance/percentage; start/end; normalized interpolation; save spline/subspline index custom data.

[S43] Custom Properties

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_operators/particles/custom_properties/

SET/GET custom data; absolute custom float values and channel names; explicit source/target data choices; default custom floats are zero.

[S44] Object Test

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_operators/particles/object_test/

Below/Above Object World-Z or Local-Z conditions and distance tests.

[S45] Operator common controls

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_operators/

Custom-float filters, On Event Entry/Continuous timing, Increment custom float test action independent of Send Out; documented conditional composition with reset channel.

[S46] Cloth Bind

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_operators/particles/cloth_bind/

Shape-to-cloth; activate/deactivate by material ID with border vertices; per-cloth mass; stretch/shear/bend/volume binding/maps; CUDA mesh collision/thickness; UV/index preservation; generated mesh.

[S47] Spline Paths

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_operators/particles/spline_paths/

Particle-binding path mode; binding ID filter; target tySplines; export ID; limits on multiple source flows.

[S48] tySplines

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_splines/tySplines/

Weld/sort bindings; linear/smooth paths, optimize controls and frame retiming; cache is RAM and not disk-persistent on its own.

[S49] tySplineMesher

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_modifiers/tySplineMesher/

Mesh mode deforms arbitrary meshes along splines; mesh/spline length conventions, repetitions, axis/pivot/orientation/twist and UV overrides.

[S50] tyParticleSkin

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_modifiers/tyParticleSkin/

Particle/mesh-driven mesh deformation; reference frame; weights by radius/falloff/max count; mesh conversion and vertex selection; element-ID matching; Weight all vertices exception; Reinitialize.

[S51] Birth Objects

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_operators/particles/birth_objects/

Mesh inheritance; split elements and pivot behavior; animation transform inheritance; save birth/element indices; explicit connection to tyParticleSkin matching.

[S52] Particle Bind Solver

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_objects/tyFlow/particleBindSolver/

Global step count, substep interaction, force integration, determinism/performance tradeoffs; cache advice.

[S53] CCCS rollout

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_objects/tyFlow/cccs/

Repulsion, impulse/IZ limits, listener warning; documented 3-5 repulsion steps and higher dense-layer guidance; impossible states can fail convergence.

[S54] Main settings

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_objects/tyFlow/mainSettings/

History dependent/independent; history-independent restarts simulation every frame; timestep; farm version consistency.

[S55] Simulation reset FAQ

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/faq/reset/

Auto mode invalidates/resimulates caches; manual mode can preserve stale states and inconsistent future results; refresh/reset required for consistency.

[S56] Particle Export

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_operators/particles/export/

Explicit export command; global/event scope; tyCache/splines/objects/Alembic/VRayProxy; cloth/spline/additional geometry options; combined meshes remove instancing; additional-geometry event scope limitation.

[S57] Rendering FAQ

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/faq/rendering/

Mesh/Trimesh as generic render geometry; Mesh needed for shape output; render-only input subdivision can trigger reset and inconsistent rendering.

[S58] Slow

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_operators/particles/slow/

Velocity damping; placement above force-adding operators; axis controls and Slow on subframes.

[S59] Simulation: Cloth/Softbodies/Ropes FAQ

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/faq/cloth/

Two official rope paths: shape spline conversion to cloth or Birth Spline plus sibling binds; Spline Paths→tySplines Weld bindings→tySplineMesher; dense cloth may require hundreds of steps.

[S60] tyPush

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_modifiers/tyPush/

Static spline pushing, Simple mode, selected knots, outward-from-meshes direction, amount/distance/falloff.

[S61] tyBend

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_modifiers/tyBend/

Manual mesh bending with axis/angle/limits and segmentation.

[S62] tyCache object

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_objects/tyCache/

Playback of exported tyCache particle/deforming mesh data.

[S63] Force

tyFlow / Tyson Ibele | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_operators/particles/force/

Built-in gravity and wind/noise; velocity/spin controls; surface-area scaling and cloth aerodynamics; optional forces use common operator filters.

[S64] Download tyFlow

tyFlow / Tyson Ibele | v2.101; CUDA modules tyFlow2.005+ CUDA12.8; official scene archive2026-09-02 | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/download/

Current downloadable labels and current CUDA package; archive availability

[S65] PhysX Shape

tyFlow / Tyson Ibele | Current unversioned manual inspected2026-10-24 | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_operators/particles/physx_shape/

Dynamic collision shapes; Mesh becomes kinematic; compound and paired-SDF limits; mass, friction, damping, locks and contact settings.

[S66] PhysX Collision

tyFlow / Tyson Ibele | Current unversioned manual | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_operators/particles/physx_collision/

Fixed mesh colliders, collision types, friction, swept contact tests and SDF limitations.

[S67] tyFlow PhysX rollout

tyFlow / Tyson Ibele | Current unversioned manual | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_objects/tyFlow/physx/

PGS/TGS, global gravity, ground collider, substeps, position/velocity iterations, CCD, sleeping and determinism.

[S68] Network rendering with tyFlow RENDER

tyFlow / Tyson Ibele | Currentlicensepage | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/license/render/

Multithreaded command-line rendering without PRO; no GPU simulation or cache export; rendering existing caches.

[S69] Mesh operator

tyFlow / Tyson Ibele | Currentmanual | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_operators/particles/mesh/

Event mesh output, combined Triangle Mesh, compatible renderer instances and render-only controls.

[S70] tyFlow Cache rollout

tyFlow / Tyson Ibele | Currentmanual;globalcacheintroduced1.019 | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_objects/tyFlow/cache/

Clear Cache/reset, RAM playback data and reuse of preprocessed mesh data.

[S71] tyMesher

tyFlow / Tyson Ibele | Currentmanual | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_objects/tyMesher/

Combined mesh, explicit frame, particle-interface choice, extracted mesh and material merging.

[S72] Collision

tyFlow / Tyson Ibele | Currentmanual | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/tyflow_operators/particles/collisions/

Scene mesh collisions, absolute radius, face sides, friction, bounce and evaluation after secondary solvers.

[S73] FAQ Motion Blur

tyFlow / Tyson Ibele | CurrentpagewitholdertyCachesubframelimit | Official documentation; accessed 24 October 2026

https://docs.tyflow.com/faq/motion_blur/

Rigid transform versus deforming-mesh motion blur; topology stability and current subframe qualification.

[S74] Using tyFlow with Corona for 3ds Max

Chaos | Official documentation; accessed 24 October 2026

https://support.chaos.com/hc/en-us/articles/4526288619793-How-to-use-tyFlow-in-Corona-for-3ds-Max

Official Chaos article updated 12 August 2026: particle interface for instancing versus ordinary mesh modifiers.

[S75] Importing SOP geometry into Solaris

SideFX | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/solaris/sop_import.html

Packed geometry import and instancing representation.

[S76] Point Instancer LOP

SideFX | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/nodes/lop/pointinstancer.html

Current dedicated LOP is marked Since 22.0; do not infer earlier UI availability.

[S77] HairOrient official example

SideFX | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/examples/nodes/sop/vellumsolver/HairOrient.html

Official asset inspected without execution; soft roots and collisions disabled in saved network.

[S78] BasicWireCapture official example

SideFX | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/examples/nodes/sop/wirecapture/BasicWireCapture.html

Official capture/deform network inspected without execution.

[S79] TwistWireDeform official example

SideFX | Official documentation; accessed 24 October 2026

https://www.sidefx.com/docs/houdini/examples/nodes/sop/wiredeform/TwistWireDeform.html

Saved tangent-plus-normal deformation frame inspected without execution.

[S80] HairOrient.hda - official downloadable asset

SideFX | Official example asset; inspected without execution

https://www.sidefx.com/docs/houdini/examples/nodes/sop/vellumsolver/HairOrient.hda

[S81] BasicWireCapture.hda - official downloadable asset

SideFX | Official example asset; inspected without execution

https://www.sidefx.com/docs/houdini/examples/nodes/sop/wirecapture/BasicWireCapture.hda

[S82] TwistWireDeform.hda - official downloadable asset

SideFX | Official example asset; inspected without execution

https://www.sidefx.com/docs/houdini/examples/nodes/sop/wiredeform/TwistWireDeform.hda

[S83] PathDeformBasic.hda - official downloadable asset

SideFX | Official example asset; inspected without execution

https://www.sidefx.com/docs/houdini/examples/nodes/sop/pathdeform/PathDeformBasic.hda

[S84] PathDeformPiece.hda - official downloadable asset

SideFX | Official example asset; inspected without execution

https://www.sidefx.com/docs/houdini/examples/nodes/sop/pathdeform/PathDeformPiece.hda

[S85] RBDBulletSolver.otl - official downloadable asset

SideFX | Official example asset; inspected without execution

https://www.sidefx.com/docs/houdini/examples/nodes/sop/rbdbulletsolver/RBDBulletSolver.otl

SECTION 14 / PRODUCT PROPOSAL

Proposed Cyrus artist workflow

This final section is a **product proposal** informed by the workflow study. It is not a description of an implemented Cyrus feature or a claim about Houdini's or tyFlow's private algorithms.

What the artist selects

The artist selects a Cyrus Scatter layer or a subset of its accepted placements, then chooses an interaction mode: **Bend rooted vegetation**, **Settle rigid objects**, or **Drape deformable leaves**. They add the relevant stones, curb and terrain to an obstacle set; a committed Scatter obstacle layer can be selected as a source of obstacle instances.

For rooted vegetation, the source asset must have a confirmed preparation record: root positions, base frames, blade membership, guides and mesh binding. Cyrus may propose roots from an axis/height rule for a simple source, but it must show that proposal for review. The artist can paint or select a root region, reverse a guide, and preview the base frame. A disconnected clump needs separate blade membership; selecting one object does not justify treating it as one rod.

For rigid objects, the artist previews the collision hull. A simple pebble defaults to a convex hull. Terrain and a curb are fixed triangle obstacles or explicit proxies. For deforming leaves, the artist confirms the segmented surface or driver mesh and sees the simulated thickness. Rigid and deforming leaf modes should visibly differ in preparation, cost and expected behavior.

Controls the artist needs

Control	Artist meaning	Proposed default / boundary
Mode	Bend, settle or drape	Explicit choice; no silent conversion from rigid to deforming.
Obstacles	Which stones/layers/ground participate	Named set with inclusion preview; all relevant obstacles evaluated together.
Clearance	Desired visible separation	Scene units, with a preview of the effective collision envelope.
Flexibility	Resistance to bending or folding	A tested source preset; do not present an uncalibrated value as a measured material property.
Root position	Whether planted roots may move	Fixed in rooted mode. A root inside an obstacle is reported.
Base orientation	Fixed frame or a soft angular attachment	Separate from root position; expose Base stiffness when Soft is chosen.
Preserve source shape	Preference for the authored rest shape	Prevent cumulative fitting from quietly redefining the source.
Collision shape	Convex, prepared compound, mesh/proxy as supported	Rigid/obstacle roles have different valid choices; include a hull preview.
Quality	Solve/contact effort and validation tolerance	A small set of qualified presets, with advanced limits available.
Update mode	Manual or update after a relevant edit	Manual initially; Live updates coalesce edits and run after the artist releases the drag.
Result controls	Update, Cancel, Reset to source, Freeze/Bake	Cache/freeze status and dependency state remain visible.

Rigid settling additionally needs gravity direction, friction, bounce and a settle-work limit. Drape needs stretch/shape resistance and optional self/inter-leaf contact. Keep those controls out of the rooted-grass panel unless they affect its selected method. Most artists should start from a source preset and adjust clearance, flexibility and base stiffness.

What Update does

1. Validate the selected source preparation and capture the current accepted placements and obstacle geometry.
2. Find affected placements using both current and previous obstacle influence, including previously bent blades that may now relax.
3. Start from the declared rest/initial state. Rooted fitting uses the authored source and base frame. Rigid settling and draping use their saved initialization poses unless the artist explicitly chooses a future **Continue from current result** operation.
4. Fit or internally settle using the selected method and a bounded work budget. The artist does not need to scrub a timeline merely to obtain a static result.
5. Validate roots/base frames, visible geometry contact, acceptable deformation and output data. A clean guide alone is insufficient.

6. Publish a complete result, update the dependency state and show counts of affected, unchanged and failed placements. Keep the previous complete publication available if the attempt is cancelled or cannot be accepted.

An internal settling calculation can legitimately sit behind one Update button. The UI should still identify the result as a settled pose when loading history matters. It should not imply that every mode has a unique history-free answer.

How failures appear

Use a persistent result status and selectable affected IDs. Distinguish **Preparation needed**, **Pending obstacle change**, **Root inside obstacle**, **Insufficient mesh resolution**, **Unresolved contact**, **Excessive deformation**, **Work limit reached**, **Cancelled**, and **Output unavailable**. Highlight the relevant blade, contact region or hull; offer the action that matches the cause.

Examples: a root-inside-stone error selects the root and stone; a binding error shows the influenced vertices and guide; a thin rigid leaf with a poor hull shows the hull overlay; an unresolved two-stone contact selects both obstacles. Do not silently move roots, delete placements, convert a rigid leaf into cloth or label a budget-limited attempt successful.

If a new attempt fails, keep the last complete result visible with an explicit pending/failed status. That geometry may no longer match the moved stone, so it must not be labelled current. The artist can lower a clearance requirement, adjust the obstacle, repair preparation, increase a justified work limit or deliberately accept a visible exception; the application should record which choice was made.

After moving a stone

In **Manual** mode, the stone moves immediately and the affected vegetation is marked Pending. The previous completed vegetation remains visible until Update. On Update, include the union of the old and new affected regions; grass left behind by the stone must be reconsidered as well as grass at its new position. Fit from the source state so repeated moves do not accumulate accidental bending.

In **Live** mode, show Pending during the drag, then schedule a bounded update after release. Replace the visible result only when a complete successor is ready. Navigation, selection and unrelated animation should not initiate a new fit.

If the stone is removed, surviving rooted blades should return toward their authored rest shape wherever no remaining obstacle requires a bend. For rigid pebbles or draped leaves, moving/removing an obstacle marks the settled result Pending and requires a fresh declared settling operation; a frozen cache remains a frozen result until explicitly updated.

For an already prepared source, the proposed initial artist sequence is: **select layer → choose mode → select obstacles → review preparation/hulls → adjust the few relevant controls → Update → inspect and save**. The normal later edit is **move stone → Update → inspect**. These are product-design targets, not measured click counts or a claim that the preparation work has disappeared.